

EchoLogz

April Carr, Jaxsen Honeycutt, Duncan Howard, Austin Johnson, and Karis Land

J.B. Speed School of Engineering, University of Louisville

CSE 350: Introduction to Software Engineering

Dr. Sai Nethra Betgeri

December 1, 2025

Table of Contents

Table of Contents	2
Brief overview/description of the project	3
Requirements	3
Functional Requirements	3
Non-Functional Requirements	7
Initial Implementation Milestones	10
Methodology	13
Need Analysis	14
Software Design	14
Risk Analysis	28
Ethics	29
Software Implementations	31
Codebase Samples	34
Test Plan	53
Test Scripts	56
Test Report	59
Team Assignments and Responsibilities	59

Brief overview/description of the project

EchoLogz is a desktop application that helps users build connections through their music tastes. The platform calculates compatibility by comparing two playlists (one from each person) and creates a musical identity for each user by analyzing key metrics within their playlists. It then generates a compatibility score, offering an engaging way to discover common ground through music.

The application is designed using React.js and Electron for the frontend UI and Python (FastAPI) for APIS is a Rust module for fast scoring. PostgreSQL is used to store user accounts and compatibility scores. Testing is implemented with Jest & React Testing Library, pytest, and Playwright.

Requirements

Functional Requirements

Functional Requirement 1.1

- FR1 – Create compare session
- ID: FR1
- Title: Start a short-lived session
- Description: The app will let a user create a session with a unique code/link to pair two users
- Priority: High
- Dependency: NA

Functional Requirement 1.2

- FR2 – Join session and paste playlist URL (public) / OR / Join session and authenticate with Spotify using OAuth 2.0
- ID: FR2
- Title: Submit public playlist URL / OR / Connect Spotify Account
- Description: Each user will join the session and paste one public Spotify playlist URL. The app validates format and reachability. / OR / Each user will join session and authenticate with Spotify using OAuth 2.0. After successful authentication, the app retrieves the user's playlists through the Spotify API. The app validates the retrieved playlists and store metadata needed for the session.
- Priority: High
- Dependency: FR1

Functional Requirement 1.3

- FR3 – Retrieve playlist tracks
- ID: FR3
- Title: Fetch playlist data
- Description: The app will fetch track IDs / metadata for both playlists via Spotify's public Web API (w/ Client Credentials if required)
- Priority: High
- Dependency: FR2

Functional Requirement 1.4

- FR4 – Persist Compatibility scores (optional)
- ID: FR4
- Title: Store pair compatibility score
- Description: If enabled, the app will store only {owner_user_id(nullable), session_id, user_a_id, user_b_id, compatibility_score, timestamp}.
- Priority: Medium
- Dependency: TBD

Functional Requirement 1.5

- FR5 – Persist Musical Identity scores (per-user, optional)
- ID: FR5
- Title: Store session outcomes
- Description: If enabled, the app will store {user_id, musical_identity_score, source_playlist_url, timestamp}.
- Priority: Medium
- Dependency: TBD

Functional Requirement 1.6

- FR6 – Aggregate users into compatibility groups
- ID: FR6
- Title: Group discovery and assignment
- Description: The app will use saved musical identity scores and compatibility scores to compute similar groups and assign each user a cluster_id. It will run community-detection on the user-score, refresh results on a schedule, and display each user's group and a brief cohort summary in the UI.
- Priority: Low
- Dependency: FR5

Functional Requirement 1.7

- FR7 – Display user profiles, matches, and profile linking with desktop app GUI
- ID: FR7
- Title: Desktop app GUI
- Description: The app frontend will be developed with React.js and Electron, as well as Python and Rust modules to implement certain APIs
- Priority: Medium
- Dependency: TBD

Functional Requirement 1.8

- FR8 – Testbed
- ID: FR8
- Title: Unit testing code and environment
- Description: Unit tests will be written for the program to verify the correctness of the code
- Priority: High
- Dependency: TBD

Functional Requirement 1.9

- FR9 – Retrieve audio features & artist genres
- ID: FR9
- Title: Enrich tracks with features
- Description: The app will fetch audio features (e.g., tempo, key, mode, energy, danceability, valence, loudness) and map artists to genres/subgenres for all tracks used in a session.
- Priority: High
- Dependency: FR2

Functional Requirement 1.10

- FR10 – User authentication & accounts
- ID: FR10
- Title: Sign up / sign in
- Description: The app will allow users to create an account and sign in (email + password or OAuth provider). Accounts will be used to associate sessions, optional storage of scores, and user settings.
- Priority: Medium

- Dependency: None

Functional Requirement 1.11

- FR11 – Visualize compatibility of users
- ID: FR11
- Title: Users Compatibility Comparison
- Description: Create a graph or heatmap showing where the two users had similar tastes and where they differed.
- Priority: Medium
- Dependency: TBD

Functional Requirement 1.12

- FR12 – Recommended songs based on other user's playlist
- ID: FR12
- Title: Song recommendation
- Description: Suggest songs based on user's top genres based on other user's playlist after comparison.
- Priority: Low
- Dependency: FR1

Functional Requirement 1.13

- FR13 – Hosted Database
- ID: FR13
- Title: Hosted Database
- Description: A hosted database will be needed to retain the information we collect
- Priority: High
- Dependency: None

Functional Requirement 1.14

- FR14 – Web Interface
- ID: FR14
- Title: Web Interface
- Description: A control panel for users to adjust account information without needing the app downloaded
- Priority: Medium
- Dependency: FR10

Non-Functional Requirements

Non-Functional Requirement 1.1

- NFR1 – Scoring latency
- ID: NFR1
- Title: Compute scores quickly
- Description: After playlists and metrics are fetched, the app will compute and render the compatibility score in ≤ 3 seconds (P95) for up to 200 tracks per playlist. End-to-end time, including API calls, should be ≤ 8 seconds (P95), in an ideal network environment.
- Priority: High
- Dependency: TBD

Non-Functional Requirement 1.2

- NFR2 – GUI aesthetics
- ID: NFR2
- Title: Ensure GUI appears responsive and sleek on target machines
- Description: Use CSS styles to ensure the app looks sleek. Include simple reactive animations and highlights for responsiveness and ensure operations do not take too long

Non-Functional Requirement 1.3

- NFR3 – Cross-Platform Compatibility
- ID: NFR3
- Title: Consistent desktop support
- Description: The application must operate consistently in Windows, macOS, and Linux environments packaged with Electron. Core features must behave identically, with no platform-specific bugs that prevent normal use.
- Priority: Medium
- Dependency: FR1-FR3

Non-Functional Requirement 1.4

- NFR4 – Authorized Login
- ID: NFR4
- Title: Login Authorization
- Description: Use OAuth for Spotify login to only provide access to authorized users.
- Priority: High

- Dependency: N/A

Non-Functional Requirement 1.5

- NFR5 – Data Privacy
- ID: NFR5
- Description: Only store necessary data, encrypt personal or sensitive data, and get user authentication when accessing non-public data.
- Priority: High
- Dependency: NFR4

Non-Functional Requirement 1.6

- NFR6 – Automated Testing
- ID: NFR6
- Title: Testability
- Description: Write modular code to implement automated unit and integration testing to quickly verify stability of new components with existing components. All code must surpass defined quality gates before integrating into codebase.
- Priority: High
- Dependency: None

Priority of Requirements

Requirement ID	Title	Requirement Type
FR10	User authentication & accounts	Functional
NFR4	Authorized login	Non-Functional
NFR5	Data privacy	Non-Functional
FR8	Testbed	Functional
FR13	Hosted database	Functional
FR3	Retrieve playlist tracks (public only)	Functional
FR1	Create compare session	Functional
FR2	Join session and paste playlist URL	Functional
FR9	Retrieve audio features & artist genres	Functional
NFR11	Scoring latency	Non-Functional
FR4	Persist musical compatibility scores	Functional
FR5	Persist musical identity scores	Functional

FR7	Display user profiles, matches, and profile linking	Functional
FR11	Visualize compatibility of users	Functional

Table 1: Priority of Requirements

Initial Implementation Milestones

Sprint 1 - HW1 ▾			
	Order	Title	State
1	▾	WEEK 4	● New
		🔒 Spotify prohibits using Spotify Data for AI training	● Resolved
		🔒 Research Functional and Non-Functional Requirements	● New
		🔒 Work on HW1-PPT	● New
		🔒 Create Presentation Video	● New
		🔒 Submit WEEK 4 Update	● Closed
2	▾	WEEK 3	● Closed
		🔒 Face2Face Agile Meeting	● Closed
		🔒 Submit WEEK 3 Update	● Closed
		🔒 Set up Discord	● Closed
3	▾	WEEK 2	● Closed
		🔒 Understand Project Expectations	● Closed
		🔒 Establish Specialized Roles	● Closed
		🔒 List Project Ideas	● Closed
		🔒 Create WEEK 3 Itinerary	● Closed
		🔒 Learn Azure DevOps	● Closed
		🔒 Create WEEK 2 UPDATE	● Closed
		🔒 Confirm WEEK 2 UPDATE	● Closed
4	▾	WEEK 1	● Closed
		🔒 Establish Group Communication	● Closed
		🔒 Set Up Azure DevOps account	● Closed
		🔒 Create GitHub Repo	● Closed

+ -		Order	Title
1	▼	📅	WEEK 9 (DUE: 10/17) + HW2
		📁	Assemble HW2 - PPT
		📁	Assemble HW2 - Presentation
2	▼	📅	WEEK 8 (DUE: 10/10)
		📁	Complete Architecture
		📁	Evaluate Risks
		📁	Create Flowchart / UML
		📁	Evaluate ethics
3	▼	📅	WEEK 7 (DUE: 10/03) + (Indiv) HW
		📁	Complete Back-End Designs
		📁	Code: Backend Skeleton
4	▼	📅	WEEK 6 (DUE: 09/26)
		📁	Complete UI designs and states
		📁	Create UI Mockups (non-programmatic)
		📁	Code: UI Skeleton
+	5	▼	📅 WEEK 5 (DUE: 09/19)
		📁	Draft scoring method - Compatibility
		📁	Draft scoring method - Musical Identity
		📁	Research existing apps and identify gaps

Sprint 3 - HW3			
		Order	Title
	+	1	WEEK 13 (DUE: 11/14) + HW3
			Write README
			Assemble HW3 - PPT
			Assemble HW3 - Presentation Video
			Record Demo Clip
			Bug Fixes
			Create test report charts
			E2E Testing
	2		WEEK 12 (DUE: 11/07)
			Code: Refactoring
			Code: Connect front-end w/ backend
			Performance tuning
			Integration Testing
			Bug Fixes
	3		WEEK 11 (DUE: 10/31)
			Finish Code Frontend: Basic UI implementation
			Finish Code Backend: Spotify fetch + scoring
			Unit testing
	4		WEEK 10 (DUE: 10/24)
			Create Test Plan
			Define test data
			Agree on quality gates
			Start Code Frontend: Basic UI implementation
			Start Code Backend: Spotify fetch + scoring

Sprint 4 - HW4 ▾	
Order	Title
1	+ - ▾ 📅 WEEKEND --> FP (DUE: 12/01) 📁 Polish FP - Documentation 📁 Polish FP - Presentation
2	+ - ▾ 📅 WEEK 15 (DUE: 11/28) 📁 Work on Final Project - Documentation 📁 Work on Final Project - Presentation 📁 Package source zip
3	+ - ▾ 📅 WEEK 14 (DUE: 11/21) 📁 Final tests & bug fixes 📁 Update & Polish README 📁 Make any changes to Flowcharts / UML, UI Mockups, etc 📁 Capture required screenshots 📁 Update Implementation Milestones timeline 📁 Finalize all code 📁 Triage open issues

Methodology

The development of EchoLogz follows an Agile methodology, promoting iterative progress, continuous feedback, and flexibility in adapting to user requirements. Work moved in cycles that each focused on expanding core functionality while steadily refining the system. This approach let the platform evolve in a controlled, intentional way, and it kept the team aligned on what needed to happen next without losing momentum.

Development began with brainstorming sessions, setting up our shared tooling, getting the GitHub repository and Azure DevOps workflows established, and defining team roles. The group also handled early constraints, including resolving Spotify's AI-training data

restriction before it could impact design decisions. This initial phase set the foundation the rest of the project relied on – communication channels were stable, collaboration patterns were set, and the vision for EchoLogz was solid enough to begin structured research and planning.

As the project moved forward, the focus shifted toward shaping the system’s design. Scoring methods were drafted, UI states and mockups were created, and an early skeleton of the interface took form. On the technical side, we built out the backend structure, finalized architectural decisions, produced UML diagrams, flowcharts, and risk and ethics evaluations. Much of this work served to lock in the core concepts so that the implementation phase had a stable blueprint to follow.

Once the design was established, development transitioned into full implementation. The team built the initial UI components, implemented the backend logic for Spotify retrieval and scoring, and set up testing expectations through a detailed test plan and defined quality gates. This period involved connecting the frontend and backend, refactoring as needed, tuning performance, and steadily moving from isolated pieces to a functional prototype. Unit testing, integration testing, and iterative bug fixes played a big role here, reinforcing stability as the system came together.

The final phase centered on refinement and preparing the complete deliverables. Code was stabilized, full end-to-end tests were run, diagrams and documentation were updated, and all project outputs were assembled.

Need Analysis

During the development of EchoLogz, we reviewed existing music-based comparison tools and identified several gaps where current solutions fall short. Our goal was to design a platform that provides clearer insights, stronger privacy, and a more data-forward experience than what is currently available.

Lack of Low-Friction Insight:

Most platforms with Spotify integrations are either social-profile add-ons or dating-app features. They offer surface-level information but no real metric of similarity. EchoLogz focuses on immediately understandable, data-driven insight — no social feed, no public profile, just a quick way to compare playlists with a friend, coworker, or potential match.

Limited Privacy Protection:

Popular options rely heavily on public social components. This creates a higher risk of data exposure or misuse. EchoLogz intentionally removes the social layer to prioritize user privacy while still delivering meaningful analytics.

Missing Quantified-Self Features:

Current tools rarely treat music taste as something measurable. EchoLogz fits naturally into the “data-about-me” culture, offering users a deeper understanding of their musical identity through metrics, patterns, and explainable comparisons.

Outdated or Incomplete Comparisons:

Dating apps like Tinder, Bumble, and Vibe display playlists or top artists but do not verify how closely two people’s listening behavior actually aligns. It’s aesthetic, not analytical. No similarity metric, no breakdown, no real comparison.

Insufficient Algorithmic Analysis:

Platforms like Musixmatch rely on crowdsourcing and lyric-focused tools. They are social and manual — not data-driven. They offer no automated compatibility scoring or algorithmic evaluation of listening patterns.

Lack of Transparency in Existing Integrations:

Spotify Blend merges two users’ tastes and gives a broad compatibility percentage, but it offers no detail on why that score exists. There are no analytics, no breakdowns, and no user-facing explanation. EchoLogz addresses this directly with clear metrics and transparent scoring.

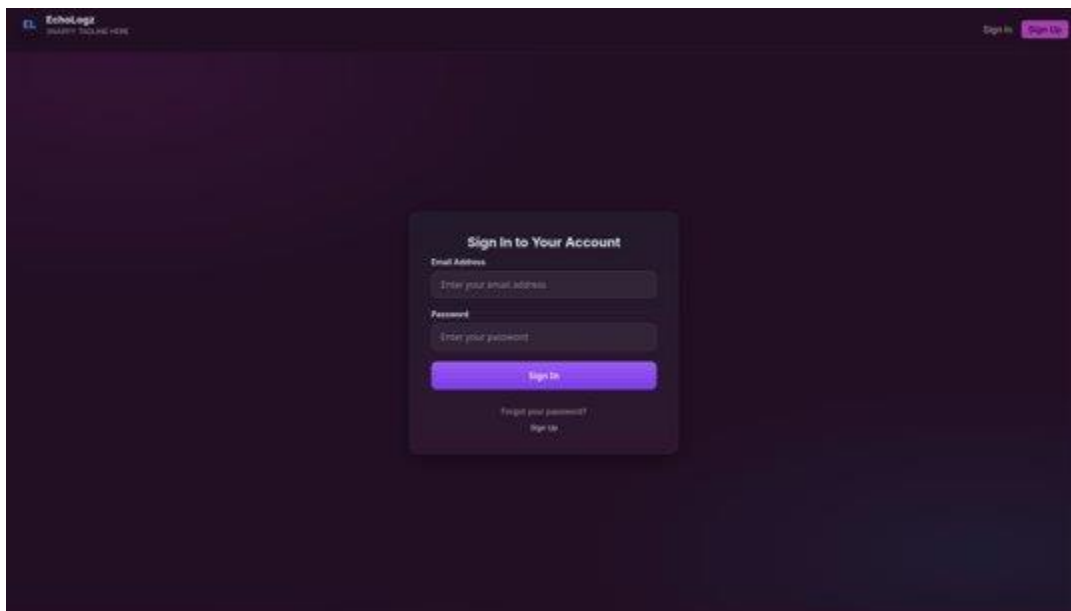
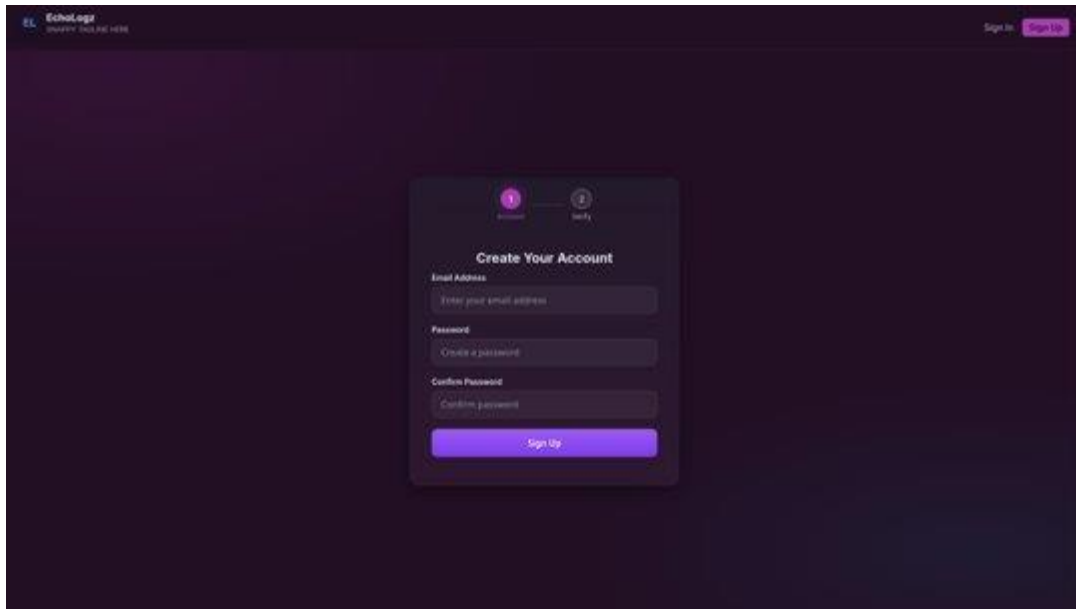
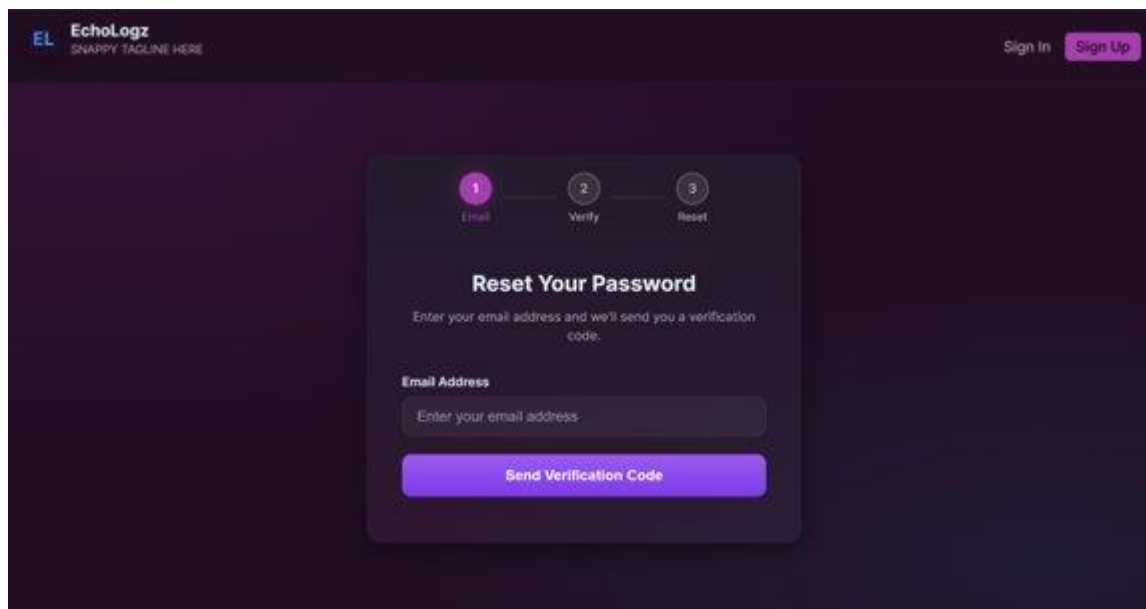
Software Design

Fig. 1: Front End Design – Login Page



The image shows a dark-themed web page for "EchoLogz" with the tagline "SNAPPY TAGLINE HERE". In the top right corner, there are links for "Sign In" and "Sign Up". The main content is a "Create Your Account" form. At the top of the form, there are two steps: "1 Email" (highlighted in red) and "2 Verify". The form fields include "Email Address" (with placeholder text "Enter your email address"), "Password" (with placeholder text "Create a password"), and "Confirm Password" (with placeholder text "Confirm password"). A red "Sign Up" button is at the bottom of the form.

Fig. 2: Front End Design – Signup Page



The image shows a dark-themed web page for "EchoLogz" with the tagline "SNAPPY TAGLINE HERE". In the top right corner, there are links for "Sign In" and "Sign Up". The main content is a "Reset Your Password" form. At the top of the form, there are three steps: "1 Email" (highlighted in red), "2 Verify", and "3 Reset". Below the steps, the text says "Enter your email address and we'll send you a verification code." The form has an "Email Address" field with placeholder text "Enter your email address" and a red "Send Verification Code" button.

Fig. 3: Front End Design – Reset Password Page

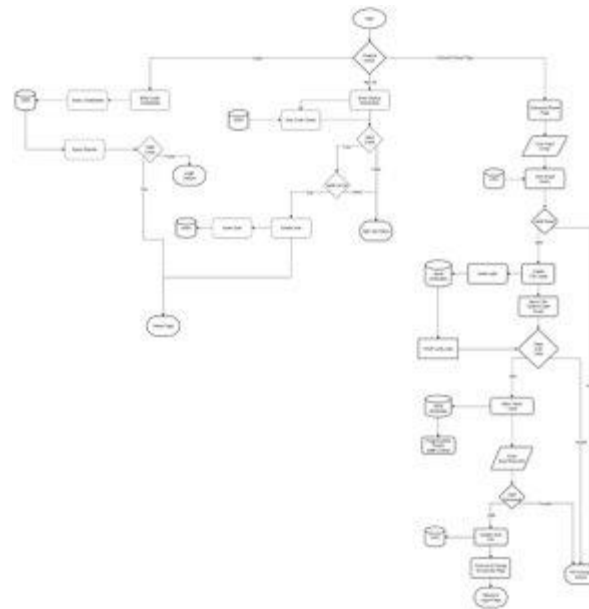


Fig. 4: Backend Design – Login, Signup, & Reset Password Page Overview

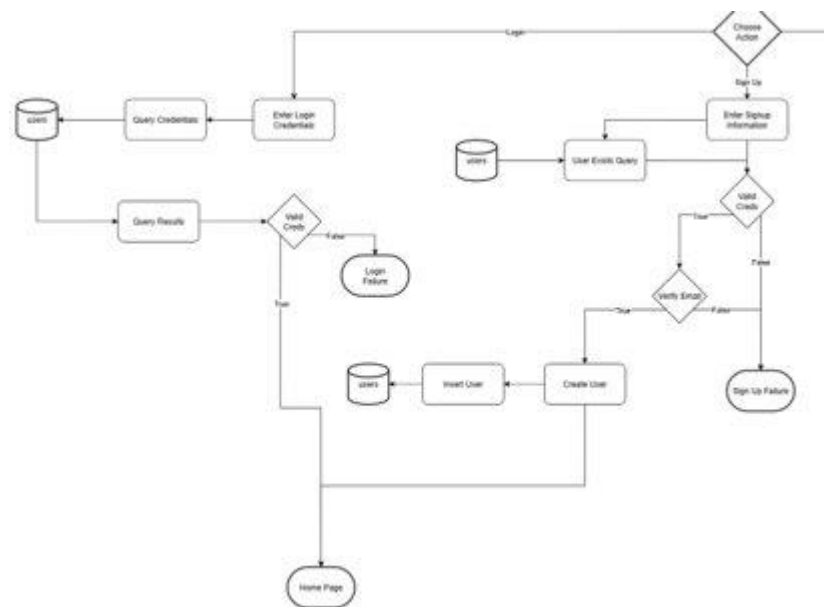


Fig. 5: Backend Design – Login & Signup

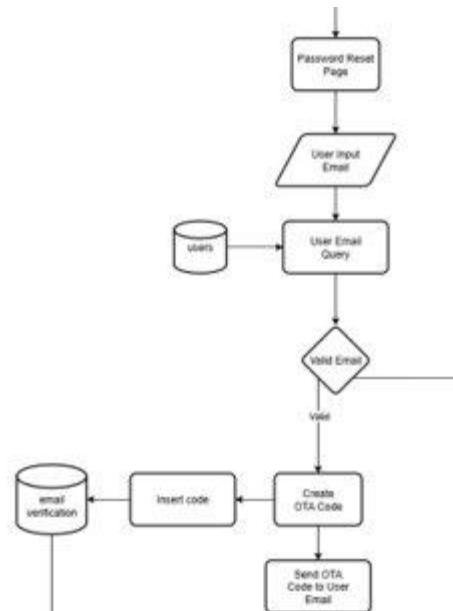


Fig. 6: Backend Design – Reset Password Page



Fig. 7: Backend Design – Reset Password Page

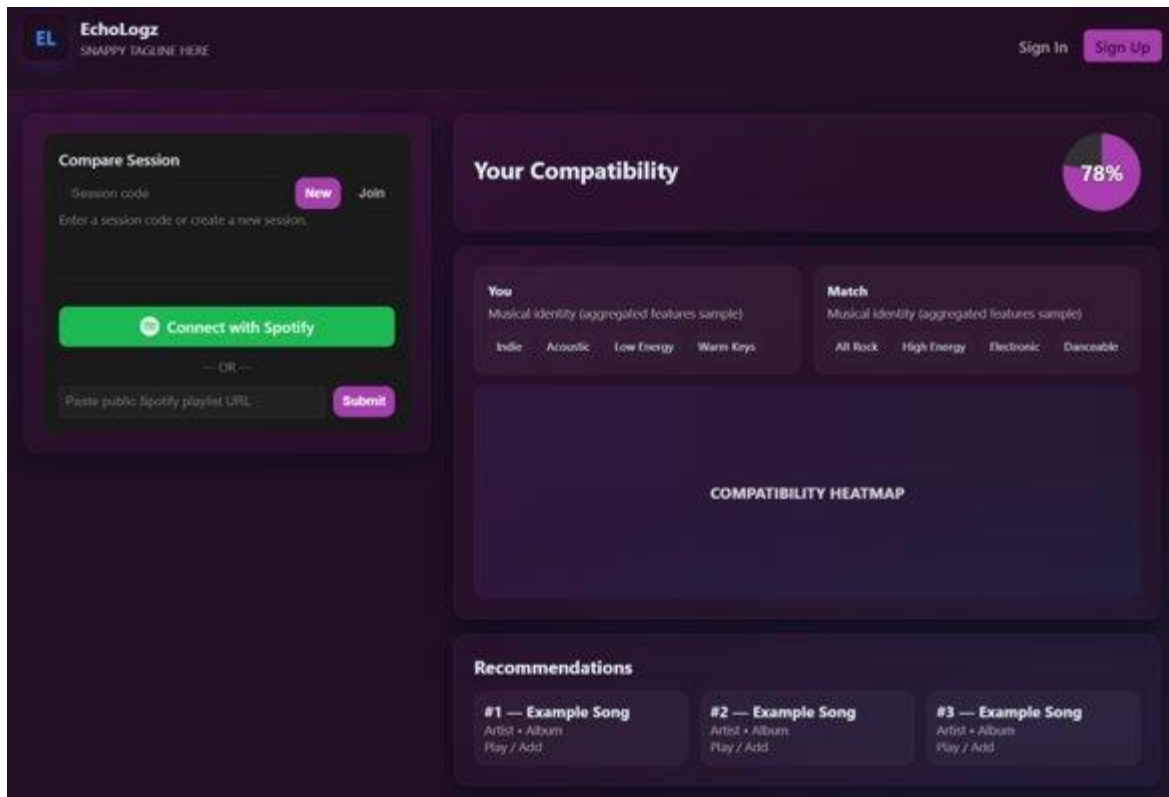


Fig. 8: Front End Design – Dashboard

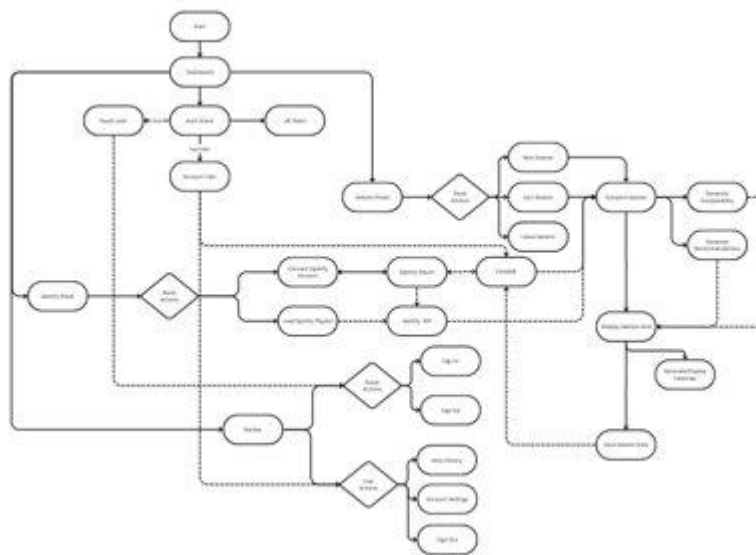


Fig. 9: Back End Design – Dashboard

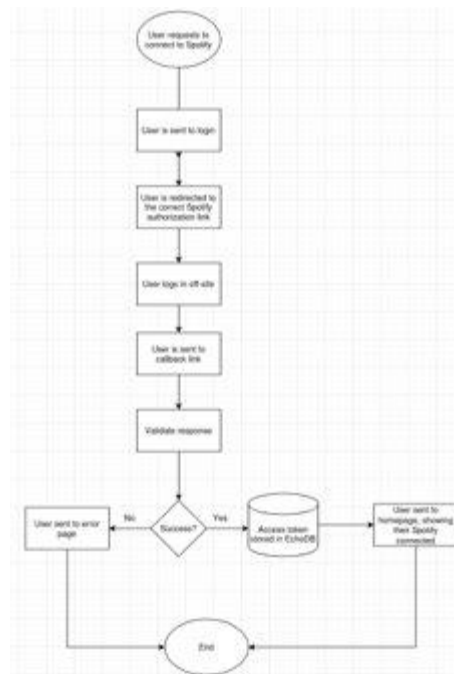


Fig. 10: Back End Design – Connect Spotify

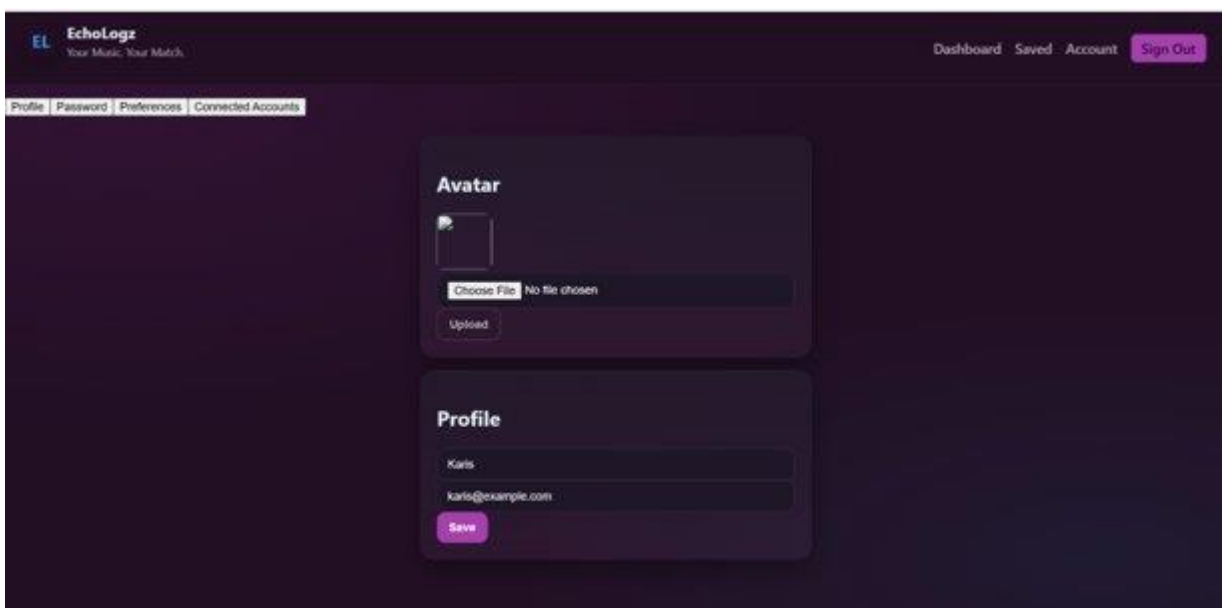


Fig. 11: Front End Design – Profile / Settings

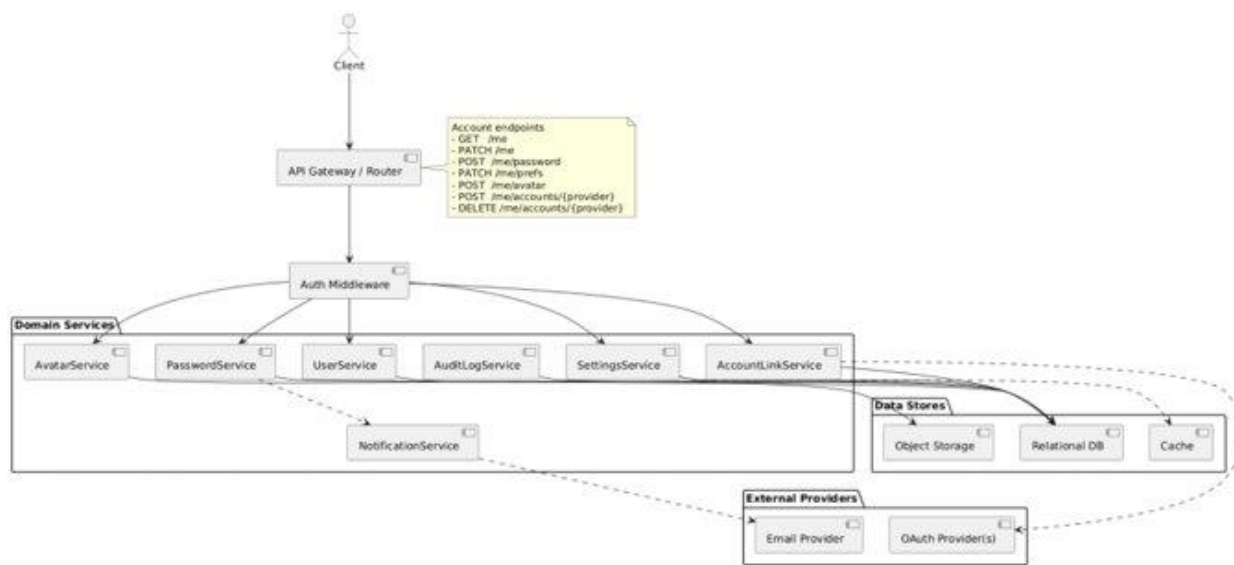


Fig. 12: Back End Design – Profile / Settings

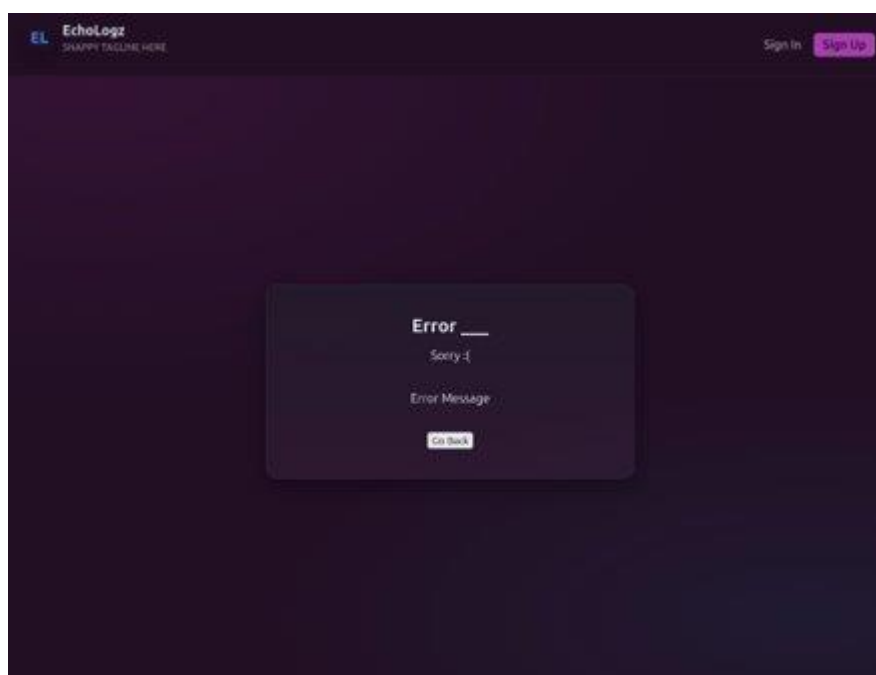


Fig. 13: Front End Design – Error

EchoLogz
History / Saved Comparisons

Search user, tag, playlist... All Status mm/dd/yyyy mm/dd/yyyy

All Me vs Others Group Playlist vs Playlist Score ≥ 0.75

Date	Participants	Type	Score	Status	Actions
2025-03-12	Karis & Sam	Me vs User	0.78	ok	View Re-run Delete
2025-03-10	Karis & Ava	Me vs User	0.64	warn	View Re-run Delete
2025-03-05	Playlist A & Playlist B	Playlist vs Playlist	0.81	ok	View Re-run Delete
2025-03-01	Team Alpha (5)	Group	0.55	err	View Re-run Delete
2025-02-26	Karis & Mina	Me vs User	0.90	ok	View Re-run Delete

6 results Prev Next

Fig. 14: Front End Design – History

EchoLogz
History / Saved Comparisons

Search user, tag, playlist... All Status mm/dd/yyyy mm/dd/yyyy

All Me vs Others Group Playlist vs Playlist Score ≥ 0.75

Date	Participants	Type	Score	Status
2025-03-12	Karis & Sam	Me vs User	0.78	ok
2025-03-10	Karis & Ava	Me vs User	0.64	warn
2025-03-05	Playlist A & Playlist B	Playlist vs Playlist	0.81	ok
2025-03-01	Team Alpha (5)	Group	0.55	err
2025-02-26	Karis & Mina	Me vs User	0.90	ok

6 results

Compare Details
Pair #3012

Date: **2025-03-12**
Participants: **Karis & Sam**
Type: **Me vs User**
Score: **0.78**
Status: **ok**

Top shared genres: **Indie, Alt Rock, Chill**
Notes: **Sample of 100 tracks per user**

Open Heatmap Re-run Delete Close

Fig. 15: Front End Design – History (Table Action: View)

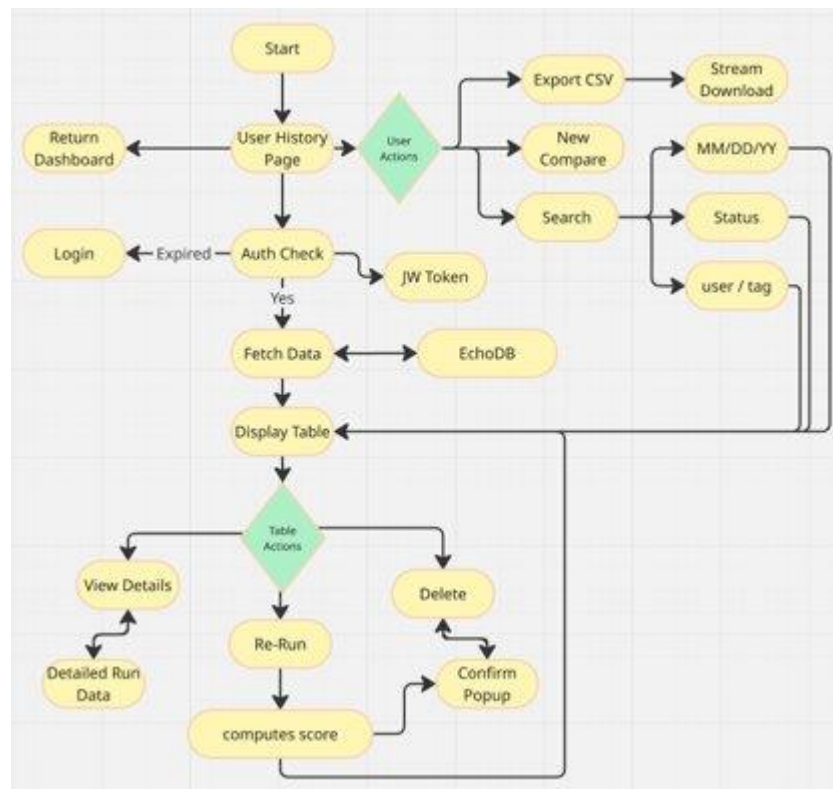


Fig. 16: Back End Design – History

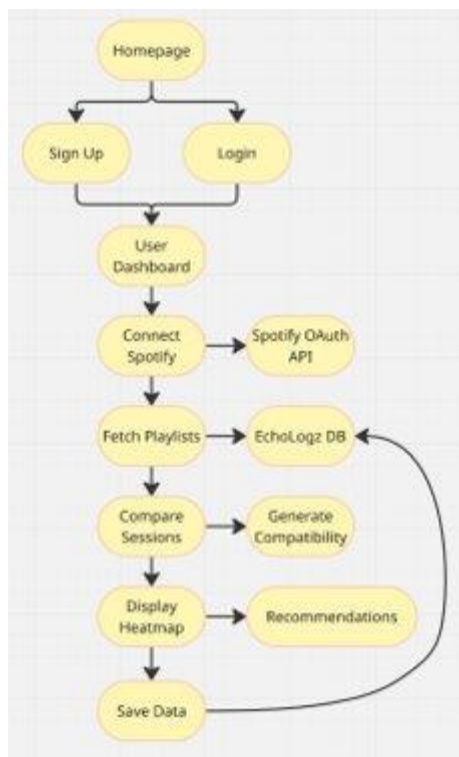


Fig. 17: User Interaction Flow

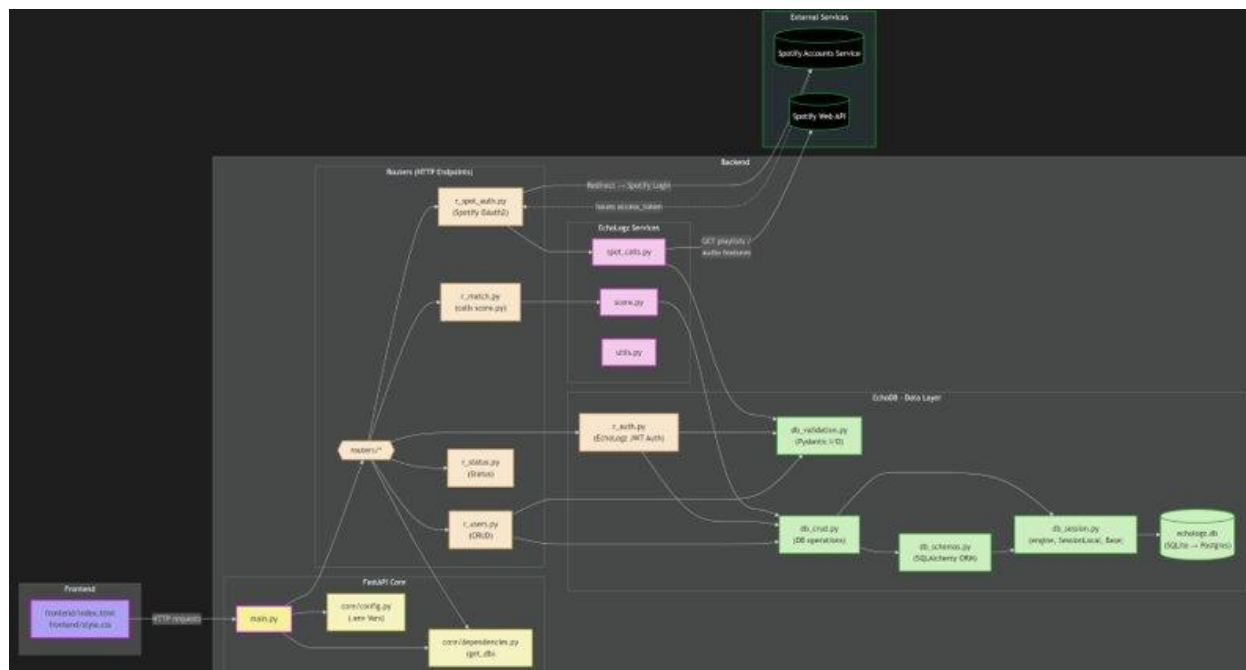


Fig. 18: High Level System Architecture

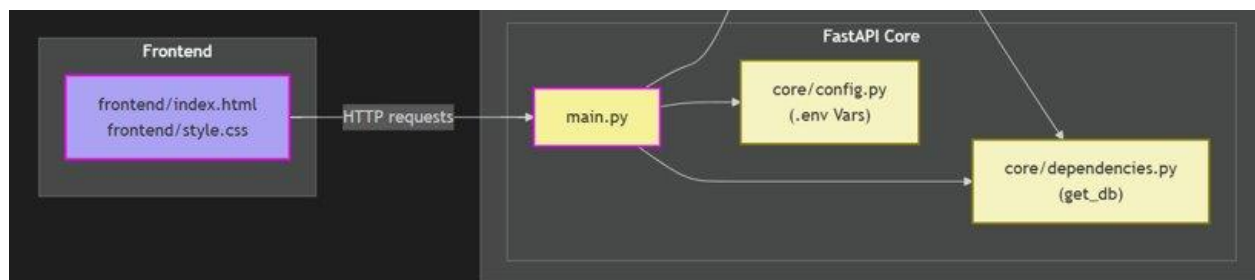


Fig. 19: High Level System Architecture (Front to Main/Core)

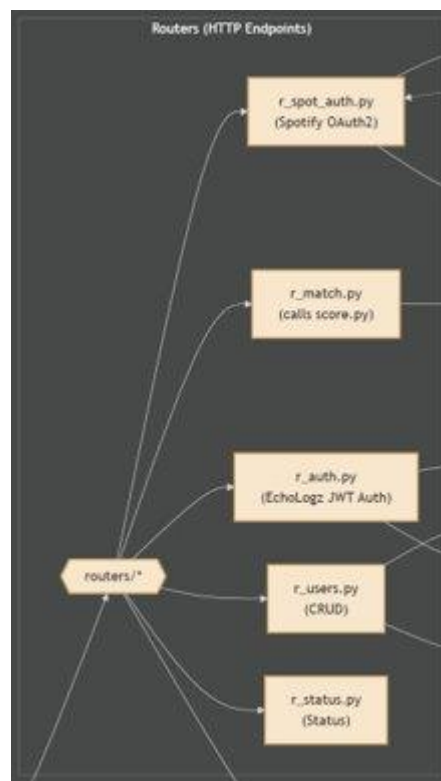


Fig. 20: High Level System Architecture (Routers)

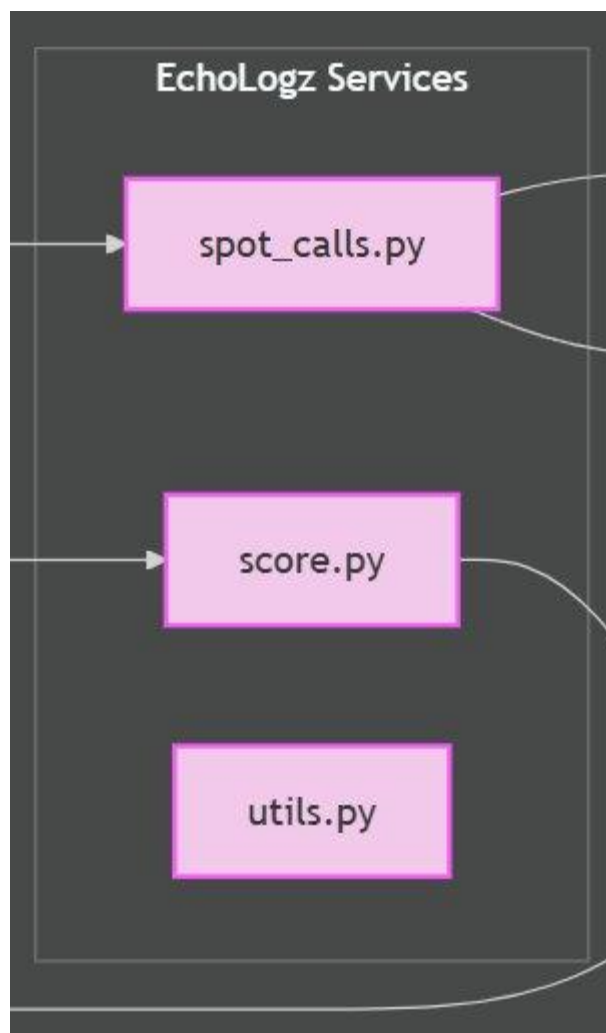


Fig. 21: High Level System Architecture (Services)

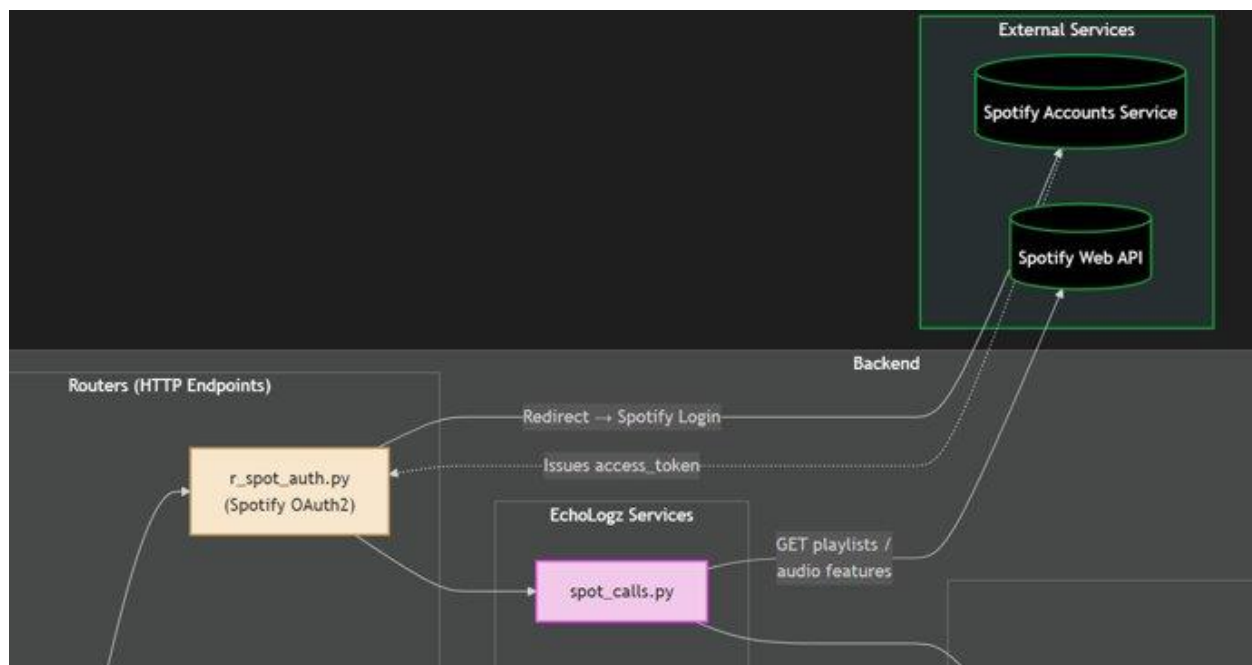


Fig. 22: High Level System Architecture (Spotify API)

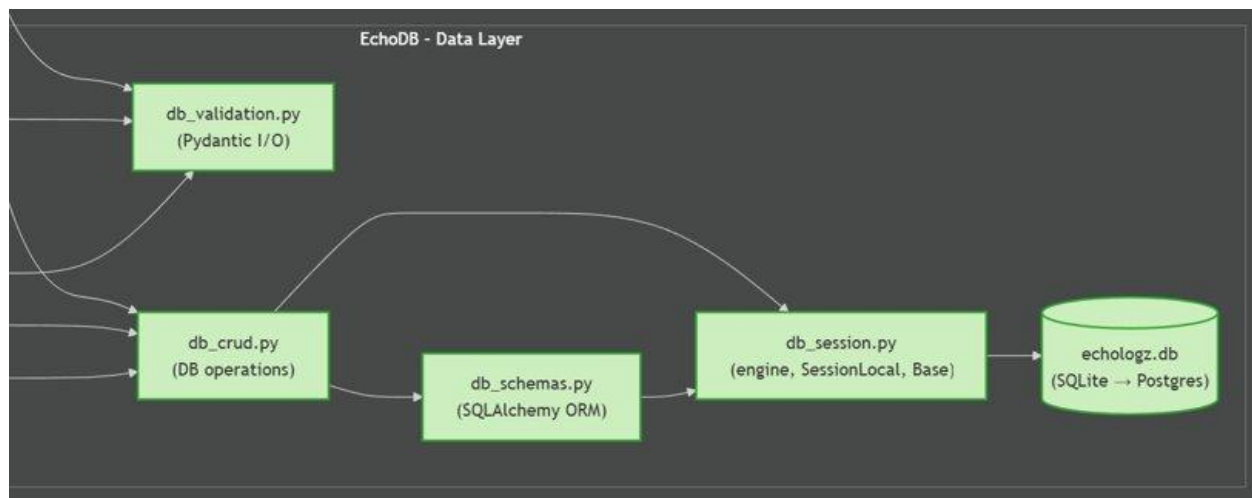


Fig. 23: High Level System Architecture (Database)

Risk Analysis

Evaluating potential risks early in development helps ensure EchoLogz remains stable, secure, and reliable as the system grows. The following risks were identified along with mitigation strategies to reduce their impact on the platform.

Data Compliance:

- **Risk:** Mishandling users' private data, including improperly stored authentication details or sensitive playlist information.
- **Mitigation:** Request explicit user consent before storing any personal data. Store all data anonymously where possible and protect sensitive information through strong encryption and secure handling practices.

Hosting Reliability:

- **Risk:** Unauthorized access through token theft, insecure storage, or weak password practices.
- **Mitigation:** Enforce HTTPS for all communication, use secure password requirements, store OAuth tokens safely with periodic rotation, and follow best practices for authentication workflows.

Data Pipeline Stability:

- **Risk:** Errors during data ingestion, transformation, or processing may lead to inconsistent or incomplete scoring data.
- **Mitigation:** Implement thorough testing for edge cases, monitor the ingestion pipeline in real time, and set automated alerts for anomalies so issues can be quickly identified and resolved.

Spotify API Rate Limits:

- **Risk:** Exceeding Spotify API rate limits may interrupt data retrieval and impact the user experience.
- **Mitigation:** Implement caching for frequently accessed data, track API response codes, and throttle requests dynamically to avoid rate limit violations

Testing & Deployment Failures:

- **Risk:** CI/CD or deployment errors may delay deliverables or cause partial outages during updates

- **Mitigation:** Use automated testing at the unit, integration, and system level to catch issues early. Configure monitoring and alerts to quickly detect deployment failures and roll back if necessary.

Scoring Algorithm Performance:

- **Risk:** Inefficient or unoptimized scoring logic may reduce performance and negatively affect the user experience.
- **Mitigation:** Optimize algorithmic complexity where possible, benchmark performance at scale, and tune backend logic to maintain fast scoring times.

Ethics

As EchoLogz handles sensitive listening data and generates personalized compatibility results, it is important to consider the ethical implications surrounding privacy, data usage, transparency, and user content. The following outlines the primary ethical concerns identified during development, along with the strategies implemented to address them.

Unauthorized Use:

- **Risk:** Third parties may attempt to misuse EchoLogz's algorithms, user data, or its Spotify API integration without proper authorization, risking data exposure or unauthorized replication of system behavior.
- **Mitigation:** Implement strong API authentication, enforce rate limits, and maintain activity logging to detect suspicious behavior. Clear and accessible terms of service will set boundaries for acceptable use.

Privacy and Data Ownership:

- **Risk:** Users' Spotify data (including listening history, playlist titles, or emotional tags) may be access, stored, or analyzed without explicit and ongoing consent.
- **Mitigation:** Use OAuth to ensure consent-based access. Anonymize any data retained for analytics and provide user-facing tools for viewing and deleting stored data. Respect ownership over their listening information at all stages of processing.

Violation of Third-Party Data Policies (Spotify Compliance):

- **Risk:** EchoLogz may unintentionally store or process Spotify-derived data beyond what is allowed by Spotify's Developer Terms, compromising compliance and user trust.
- **Mitigation:** Cache only the minimum temporary data required for active sessions and avoid storing full playlists or long-term listening histories. Review Spotify's

Developer Policies regularly, delete retained user data after session completion, and use automated purge mechanisms for cached content.

Consent Drift:

- **Risk:** Data originally shared for one purpose (such as playlist comparison) may later be reused for unrelated features without renewed permission, leading to ethical misuse.
- **Mitigation:** Restrict all data usage to the specific purpose described at the moment of consent. Prompt users for new consent before expanding to additional features or analytical uses.

Algorithmic Misrepresentation (Scientific Validity & Transparency):

- **Risk:** Compatibility results could be interpreted as scientifically validated or psychologically accurate when they are, in fact, heuristic and designed for entertainment or musical exploration. This may cause users to incorrectly view results as meaningful assessments of relationships, identity, or personality.
- **Mitigation:** Provide clear disclaimers indicating that compatibility metrics are interpretive and not scientific. Explain algorithmic assumptions, limitations, and intended use cases. Consider transparency reports outlining how scores are calculated to avoid misrepresentation.

Ethical Guidelines Followed in the Project:

- **Respect for Ownership:** EchoLogz relies only on open-source or properly licensed frameworks and libraries. Spotify account data is accessed exclusively through OAuth, and Spotify's data retention policies are documented directly within relevant modules.
- **Avoiding Plagiarism:** All core features were implemented by team members. Industry standards and common design conventions are used throughout the project, and any borrowed code snippets are credited appropriately.
- **Confidentiality:** EchoLogz never stores Spotify passwords. OAuth tokens, JWTs, and hashed credentials safeguard user data. Full data-deletion options allow users to remove all associated metadata at any time.
- **Promoting Diversity:** EchoLogz is intentionally designed to explore a wide range of musical tastes, emphasizing inclusivity and connection through music irrespective of cultural, demographic, or genre-based boundaries.

EchoLogz Spotify Data Rules:

- **Allowed to Store:** Spotify user ID, access/refresh tokens, expiration times, granted scopes, last-sync timestamps, compact derived vectors, and select IDs needed for feature implementation.
- **Not Allowed to Store:** Spotify passwords, authorization codes post-use, raw audio files, full listening histories, complete playlist dumps, or any data outside what is required for EchoLogz features.
- **Required Behavior:** Use only approved scopes, never log tokens in plain text, avoid exposing credentials to the frontend, and delete all Spotify-derived data upon user disconnect.

Software Implementations

The implementation of EchoLogz is built on a full-stack architecture designed to be lightweight, fast, and scalable while keeping costs at zero for development and testing. The system is divided into a React-based frontend, a FastAPI backend, and a PostgreSQL database layer, supported by a toolchain of authentication libraries, API clients, caching utilities, and numerical computation packages. Each component of the stack plays a specific role in how the platform processes Spotify user data, generates compatibility vectors, and presents results through an intuitive interface.

Frontend Implementation:

The frontend is written in **React.js (v18)** using **JavaScript ES6+** and **JSX**, which provides a dynamic, component-based foundation for the EchoLogz interface. Core markup and styling rely on **HTML5**, **CSS3**, and responsive design principles to maintain consistency across screen sizes. Fonts are served through **Google Fonts (Inter)** to give EchoLogz a clean, modern visual identity.

ReactDOM handles rendering components into the DOM, while optional styling tools such as CSS modules or Styled Components allow panels and widgets to remain visually consistent without leaking styles across the app. For prototyping, the system can be run through a CDN like **unpkg**, though the final implementation uses local builds bundled by Vite.

Overall, the frontend manages:

- user-facing UI panels,
- protected routing after authentication,

- Spotify Connect workflow screens,
- test result displays,
- and dynamic components for comparing taste vectors.

Backend Implementation:

The backend is implemented using **Python 3.12** with **FastAPI** as the primary API framework. FastAPI provides fast routing, type-safe request handling, and automatic documentation generation, which made it well suited for EchoLogz's modular structure.

For data modeling, the backend uses **SQLAlchemy** with **Alembic** for migrations, enabling a structured relational model for user accounts, Spotify tokens, session logs, and derived analytical data. SQLite is used for local development, while **PostgreSQL** is configured for production-level deployments.

Spotify authentication and data fetching are handled through **Authlib** and direct **Spotify Web API** integration. Access and refresh tokens are stored securely, with expiration times and scopes recorded to enforce Spotify's data retention rules.

Backend responsibilities include:

- OAuth 2.0 authorization flow,
- ingestion and cleaning of Spotify listening data,
- generation of numerical taste vectors through **NumPy/SciPy**,
- compatibility scoring,
- Spotify rate-limit handling,
- and secure delivery of processed results to the frontend.

Caching, Performance, and Runtime:

For improved efficiency, EchoLogz uses **Redis** to temporarily cache frequently accessed Spotify data and session metadata, reducing unnecessary API calls. Requests are served using **Uvicorn**, the ASGI runtime responsible for handling concurrency and keeping API response times low.

This infrastructure allows EchoLogz to perform vector math operations (cosine similarity and related measures) quickly, while also being able to scale horizontally if deployed in a cloud environment.

Integration Between Frontend and Backend:

Communication between the React frontend and FastAPI backend occurs over RESTful

endpoints protected by authenticated headers. Frontend requests trigger backend ingestion of Spotify data, scoring operations, and persistence into the relational database. The backend responds with structured JSON objects containing compatibility metrics, match insights, and user-specific vector data.

Debugging and Issue Resolution

Throughout development, several issues were identified during UI integration, authentication workflow testing, and frontend–backend communication. The following summarizes key bugs encountered during implementation and the steps taken to resolve them:

Inaccurate Password Validation Message:

- **Issue:** When users submitted a password longer than eight characters on the Sign Up page, the system consistently returned the error *“Password must be at least 8 characters.”*
- **Cause:** The validation logic correctly enforced three requirements — minimum length, at least one number, and no whitespace — but the error message only reflected the first criterion.
- **Resolution:** The error text was updated to describe all password requirements, ensuring users receive accurate guidance and reducing failed sign-up attempts.

Frontend Failing to Communicate With Backend During Sign-Up:

- **Issue:** Selecting “Submit” on the Sign Up page failed to trigger any request to the API, leaving users unable to create an account.
- **Cause:** A placeholder function was still active in the submission handler and never forwarded data to the working backend endpoint.
- **Resolution:** The placeholder handler was removed and replaced with a functional API call tied to the correct FastAPI route, restoring full sign-up capability.

Input Fields Blocking User Typing:

- **Issue:** Users could only enter a single character into input fields before losing focus, making every form unusable.
- **Cause:** The React build configuration was incompatible with the behavior of the development server, causing React to re-render unpredictably on each keystroke.

- **Resolution:** A fresh Vite base application was generated, and existing components were migrated into the Vite project structure. This resolved the re-rendering issue and restored stable input handling.

Note on Spotify API Feature Access:

During the final stages of development, Spotify fully removed access to several track-level audio feature endpoints that had previously been marked as deprecated. Historically, “deprecated” in Spotify’s API ecosystem indicated future removal but did not block requests outright. As of late Fall 2025, these endpoints now return authorization errors even when proper scopes and tokens are provided.

This change occurred mid-project and prevented EchoLogz from retrieving audio features for tracks, which had originally been planned as part of the scoring pipeline. The system was adapted to rely on alternate, non-deprecated metadata and derived vector components, allowing the core comparison and compatibility features to remain functional without violating Spotify’s updated API policies.

Codebase Samples

```

"""
Main Application Entry Point

This module initializes and runs the EchoLogz FastAPI backend.
It handles the following core responsibilities:

- Creates the FastAPI app instance with the project title.
- Imports and initializes the database and ORM models.
- Automatically creates database tables (if they don't already exist).
- Defines basic API routes (starting with a simple root health check).

NOTE: CORSMiddleware
# ... CORS: Allows communication btwn diff ports (frontend -> backend) - which is only issue during dev
# ... FRONT-END (DEV): 127.0.0.1:5500 --> BACK-END (DEV): 127.0.0.1:8000
# ... FRONT-END (DEPLOY): https://echologz(or whatever).com --> BACK-END (DEPLOY): https://echologz(or
whatever)/api.com

Run with Terminal Command:
| | | uvicorn backend.main:app --reload

Access the running server at:
| | | http://127.0.0.1:8000/
| | | http://127.0.0.1:8000/docs <-- Swagger UI (FastAPI Interactive API Documentation)

"""

```

Fig. 24: Back End – Main

```

# INTERNAL IMPORTS:
from backend.echoDB import db_tables
from backend.routers import r_auth, r_spot_auth, r_match, r_users
from backend.echoDB import db_session
# from backend.core.config import settings # Load (.env) variables via config.py

# EXTERNAL IMPORTS
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from starlette.staticfiles import StaticFiles
#-----

# CREATE ALL DB TABLES IF THEY DON'T ALREADY EXIST IN ECHOLOGZ.DB
@asynccontextmanager
async def lifespan(app: FastAPI): # Runs when the app starts
    db_tables.Base.metadata.create_all(bind=db_session.engine)
    yield # Runs when the app stops (if you need cleanup)

app = FastAPI(title="EchoLogz API", lifespan=lifespan)
# app.mount("/static", StaticFiles(directory="static"), name="static")
# Static file serving disabled (KL): No static assets needed-React frontend runs separately.

```

Fig. 25: Back End – Main

```

# CORS CONFIG:
app.add_middleware(
    CORSMiddleware,
    allow_origins=[ # ALLOW FRONT-END ---> HTTP REQUESTS ---> BACK-END: 127.0.0.1:8000
        "http://127.0.0.1:5500", "http://localhost:5500",
        "http://127.0.0.1:3000", "http://localhost:3000",
        "https://echoquest.app" # production frontend
    ],
    allow_credentials=True,
    allow_methods=["*"], # or ["GET", "POST"] if you want to limit
    allow_headers=["*"], # or ["Authorization", "Content-Type"]
)

# ROUTERS:
app.include_router(r_auth.router)
app.include_router(r_spot_auth.router)
app.include_router(r_users.router)
app.include_router(r_match.router)
# app.include_router(r_status.router)

# TEST ROUTE:
@app.get("/")
def read_root():
    return {"message": "EchoLogz backend is running!"}

```

Fig. 26: Back End – Main

```
-----
MODULE: EchoLogz Auth Router (r_auth.py)
-----
```

Responsibilities:

Handles all authentication endpoints for the EchoLogz backend.

Provides login, token generation, and user verification using a database-backed (persistent data, not temporary) user store.

Uses JWT for stateless authentication (no server-side session storage).

All request/response schemas come from backend.echoDB.db_schemas.

All database access is handled through SQLAlchemy sessions and CRUD utilities.

Fig. 27: Back End – FastAPI Routers | AUTH

```
@router.post("/signup", response_model=UserOut, status_code=201)
def signup(payload: UserCreate, db: Session = Depends(get_db)):
    """Creates a new user account.
    - validates request body, check username exists, hashes pw, inserts user -> DB
    Returns: user info sans pw (UserOut shaped)
    """
    if db_crud.get_user_by_username(db, payload.username):
        raise HTTPException(status_code=400, detail="Username is taken")
    hashed = security._hash_password(payload.password)
    user = db_crud.create_user_with_hash(
        db=db,
        username=payload.username,
        email=getattr(payload, "email", None),
        hashed_pw=hashed,
    )
    return UserOut.model_validate(user)
```

Fig. 28: Back End - FastAPI Routers | AUTH

```
-----  
MODULE: Compatibility Match Router (r_match.py)  
-----
```

This router defines the API endpoints for user-to-user or entity-to-entity matching within the EchoLogz ecosystem.

It serves as the entry point for frontend clients (or other services) to request comparison and compatibility scoring between users based on their Spotify data or other stored features (in EchoLogz DB).

Core Responsibilities:

- ```

```
- Accept and validate incoming match requests (user IDs, sample size, etc.)
  - Pass validated data to the scoring engine (services.score)
  - Return standardized match results containing overall and sub-scores
  - Handle input validation and error responses for failed or invalid comparisons

Purpose:

```

```

Acts as the gateway between the EchoLogz frontend and the internal scoring system.

*Fig. 29: Back End - FastAPI Routers | MATCH*

```

@router.post("/compare", response_model=CompareResp)
def post_compare(req: CompareReq, db: Session = Depends(get_db)):
 """ Compare two users by their feature vectors and return a compatibility score.
 -> build_feature_vector(db, user_id=1)
 -> look up user 1 in EchoLogz DB
 -> find spotify_account_id (not implemented)
 -> find access_token / refresh_token (not implemented)
 -> call Spotify API to get features (not implemented)
 -> convert features to a numeric vector
 -> build_feature_vector(db, user_id=2)
 -> calc_compatibility(vec_a, vec_b)
 -> return score
 """
 try:
 user_a_vec = build_feature_vector(
 db=db,
 user_id=req.user_a_id,
 sample=req.sample,
)
 user_b_vec = build_feature_vector(
 db=db,
 user_id=req.user_b_id,
 sample=req.sample,
)
 score = calc_compatibility(user_a_vec, user_b_vec) # Compute compatibility
 pair_id = None # Look up / store a pair_id (?)
 return CompareResp(score=score, pair_id=pair_id)
 except ValueError as e: # bad-input - features or math
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail=str(e),
)

```

Fig. 30: Back End - FastAPI Routers | MATCH

```

MODULE: Spotify OAuth Router - The Gatekeeper (r_spot_auth.py)

```

Handles the OAuth dance with Spotify:

- /auth/spotify/login -> Redirect user to Spotify consent
- /auth/spotify/callback -> Exchange code for tokens
- (optional) /auth/spotify/refresh -> Refresh access token

Secure storage notes:

- Never return raw Spotify tokens to the client.
- Encrypt/secure tokens at rest - store refresh\_token to renew access.
- Tie Spotify tokens to the EchoLogz user (foreign key to your users table).

*Fig. 31: Back End - FastAPI Routers | SPOT AUTH*

```
@router.get("/login")
def login_spotify():
 params = {
 "client_id": SPOTIFY_CLIENT_ID,
 "response_type": "code",
 "redirect_uri": SPOTIFY_REDIRECT_URI,
 "scope": SCOPES,
 # "state": "...", # Optional: add CSRF protection
 # "show_dialog": "true",
 }
 url = "https://accounts.spotify.com/authorize?" + urlencode(params)
 return RedirectResponse(url)
```

*Fig. 32: Back End - FastAPI Routers | SPOT AUTH*

```

@router.get("/callback")
def spotify_callback(
 code: str,
 db: Session = Depends(get_db),
 current_user: db_tables.User = Depends(get_current_spotify_account),
):
 token_url = "https://accounts.spotify.com/api/token"
 data = {
 "grant_type": "authorization_code",
 "code": code,
 "redirect_uri": SPOTIFY_REDIRECT_URI,
 }
 headers = _basic_auth_header(SPOTIFY_CLIENT_ID, SPOTIFY_CLIENT_SECRET)
 resp = requests.post(token_url, data=data, headers=headers)
 tokens = resp.json()
 if "access_token" not in tokens:
 raise HTTPException(400, tokens.get("error_description", "Failed to get token"))
 access_token = tokens["access_token"]
 refresh_token = tokens.get("refresh_token")
 expires_in = tokens["expires_in"]
 me = requests.get(
 "https://api.spotify.com/v1/me",
 headers={"Authorization": f"Bearer {access_token}"},
).json()
 spotify_user_id = me["id"]
 expires_at = datetime.now(timezone.utc) + timedelta(seconds=expires_in)
 db_crud.create_or_update_spotify_account(
 db=db,
 user_id=current_user.id,
 spotify_user_id=spotify_user_id,
 access_token=access_token,
 refresh_token=refresh_token,
 expires_at=expires_at,
 scope=SCOPES,
)
 return {"message": "Spotify connected"}

```

Fig. 33: Back End - FastAPI Routers | SPOT AUTH



```

MODULE: User Router (r_users.py)

```

Defines the API routes under "/users" for EchoLogz app users.

What this module DOES:

- Declare HTTP endpoints (list, create, read, update, delete).
- Delegate DB work to echoDB.db\_crud.
- Validate I/O with echoDB.db\_schemas Pydantic models.
- Use dependency injection to get a DB session.

What this module DOES NOT do:

- No raw SQL or ORM session management (see echoDB.db\_session).
- No table/model definitions (see echoDB.db\_tables).
- No Spotify OAuth or API calls (see routers/spotify\_auth.py and services/spotify\_calls.py).
- No business rules beyond simple request → service delegation.

Outcome: Thin routing layer that converts HTTP requests into db\_crud calls and returns clean JSON responses.

*Fig. 34: Back End - FastAPI Routers | USERS*

```

router = APIRouter(prefix="/users", tags=["users"])

@router.post("/", response_model=val.UserOut, status_code=status.HTTP_201_CREATED)
def create_user_endpoint(payload: val.UserCreate, db: Session = Depends(get_db)):
 """Create a new user and return the created record."""
 return crud.create_user(db, payload)

@router.get("/{user_id}", response_model=val.UserOut)
def get_user_endpoint(user_id: int, db: Session = Depends(get_db)):
 """Return a single user by id, or 404 if not found."""
 user = crud.get_user_by_id(db, user_id)
 if not user:
 raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found")
 return user

@router.get("/", response_model=list[val.UserOut])
def list_users_endpoint(db: Session = Depends(get_db)):
 return crud.list_users(db)

@router.put("/{user_id}", response_model=val.UserOut)
def update_user_endpoint(
 user_id: int,
 payload: val.UserUpdate,
 db: Session = Depends(get_db),
):
 """ Replace an existing user with the provided fields.
 (All fields in UserUpdate are optional; absent fields are not changed.) """
 user = crud.update_user(db, user_id, payload)
 if not user:
 raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found")
 return user

@router.delete("/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_user_endpoint(user_id: int, db: Session = Depends(get_db)):
 """Delete a user; returns 204 if successful, 404 if not found."""
 ok = crud.delete_user(db, user_id)
 if not ok:
 raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found")
 # Explicitly return an empty body with 204
 return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Fig. 35: Back End - FastAPI Routers | USERS

```

"""

MODULE: Database ORM Models: DB Blueprint - ORM table models (db_tables.py)

This module defines the database schema for EchoLogz using SQLAlchemy ORM.
Each model/class represents a table in the database and maps Python objects
to relational database records.

NOTE: SQLAlchemy ORM models (Object-Relational Mapping models) are Python classes that
represent tables in a relational database. Each class corresponds to one table, and each
class attribute corresponds to a database column. The ORM automatically translates
between Python objects and SQL queries, so you can work with database rows using normal
Python code instead of writing raw SQL statements.
NOTE: ORM models are *pythonic schemas* that are also mapped directly to actual database
tables – meaning they act as both the blueprint and the concrete table representation at
once.

Responsibilities:
- Defines table structures and their columns (schemas).
- Maps Python classes to relational database tables using SQLAlchemy ORM.
- Establishes relationships (foreign keys, joins) between tables.
- Inherits from Base (provided by db_session.py) to register models with SQLAlchemy.

Modules using db_tables.py:
- db_crud.py
- db_session.py

INTERNAL IMPORTS:
- db_session.py -> Provides the SQLAlchemy Base class used for model inheritance.

EXTERNAL IMPORTS:
- SQLAlchemy

NOTE: Spotify Developer Terms = "Except as otherwise set out in these Developer Terms,
you may not store, aggregate or create compilations or databases of Spotify Content,
other than as strictly necessary to operate your SDA."
WORK AROUND: Store Track IDs + URIs (this is ok)
"""

```

*Fig. 36: Back End – DB Tables*

```
=====
EchoLogz User Table
=====

class User(Base):
 __tablename__ = "users"
 id = Column(Integer, primary_key=True, index=True)
 username = Column(String, unique=True, nullable=False)
 email = Column(String, unique=True, nullable=False)
 hashed_password = Column(String, nullable=False)

 # Link to SpotifyAccount rows
 spotify_accounts = relationship(
 "SpotifyAccount",
 back_populates="user",
 cascade="all, delete-orphan",
 passive_deletes=True,
)
```

Fig. 37: Back End – DB Tables

```
=====
Spotify User Table
=====

class SpotifyAccount(Base):
 __tablename__ = "spotify_accounts"
 id = Column(Integer, primary_key=True, index=True)
 user_id = Column(
 Integer,
 ForeignKey("users.id", ondelete="CASCADE"),
 nullable=False,
 index=True,
) # FK to EchoLogz users table

 spotify_user_id = Column(String, nullable=False, index=True) # Spotify's stable user identifier
 access_token = Column(String, nullable=False) # OAuth2 token bundle
 refresh_token = Column(String, nullable=False)
 expires_at = Column(DateTime, nullable=False) # When the access token expires (UTC)
 scope = Column(String, nullable=True) # Optional: space-separated scopes requested
 last_synced_at = Column(DateTime, nullable=True, default=None) # Bookkeeping: Last time we pulled data
 from Spotify
 user = relationship(
 "User",
 back_populates="spotify_accounts",
) # ORM relationship back to User
```

Fig. 38: Back End - DB Tables

```

INTERNAL IMPORTS:
from . import spot_calls
from backend.echoDB import db_crud

EXTERNAL IMPORTS:
from sqlalchemy.orm import Session
import numpy as np
from datetime import datetime, timezone
from backend.services.utils import normalize_vector

def build_feature_vector(db: Session, user_id: int, sample: str = "medium_term") -> list[float] | None:
 # DB + Spotify + high-level orchestration

 # Step 1: Lookup EchoLogz user (assumes valid user_id given)
 user = db_crud.get_user_by_id(db, user_id)
 if not user:
 return None

 # Step 2: Lookup Spotify link
 spotify_account = db_crud.get_spotify_account(db, user_id)
 if not spotify_account or not spotify_account.access_token:
 return None

 # Step 3: Token lifecycle management
 if _token_is_expired(spotify_account):
 new_token, new_expiry = spot_calls.refresh_access_token(spotify_account.refresh_token)
 if not new_token:
 return None
 spotify_account.access_token = new_token
 spotify_account.token_expires_at = new_expiry
 db.commit()

```

*Fig. 39: Back End | Services – Spot Feature Vector*

```

 access_token = spotify_account.access_token

 # Step 4: Get Spotify tracks for the user
 track_ids = spot_calls.get_user_top_tracks(access_token, time_range=sample)
 if not track_ids:
 return None

 # Step 5: Get audio features
 raw_features = spot_calls.get_audio_features(access_token, track_ids)
 if not raw_features:
 return None

 # Step 6-7: Reduce to taste vector and return
 return _matrix_features_to_vector(raw_features)

def _matrix_features_to_vector(raw_features: dict[str, dict[str, float]]) -> list[float]:
 # math only

 if not raw_features:
 return []

 matrix = np.array([list(track.values()) for track in raw_features.values()])
 mean_vector = matrix.mean(axis=0)
 return normalize_vector(mean_vector).tolist()

def _token_is_expired(spotify_account) -> bool:
 return spotify_account.token_expires_at < datetime.now(timezone.utc)

```

Fig. 40: Back End | Servies - Spot Feature Vector

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <title>Account - EchoLogz</title>
6 <link rel="stylesheet" href="style.css" />
7 <link rel="stylesheet" href="pw-style.css" />
8 <!-- React, ReactDOM, Babel -->
9 <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
10 <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
11 <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
12 </head>
13 <body>
14 <div id="root"></div>
15
16 <script type="text/babel" src="js/navbar.jsx"></script>
17 <script type="text/babel">
18 const { useState, useEffect, useRef } = React; /* ----- UI Feedback (toast) ----- */
19
20 function Toast() {
21 const [msg, setMsg] = useState("");
22 const timerRef = useRef(null);
23 useEffect(() => {
24 window.UIFeedback = {
25 show: (m, ms = 1800) => {
26 setMsg(m);
27 if (timerRef.current) clearTimeout(timerRef.current);
28 timerRef.current = setTimeout(() => setMsg(""), ms);
29 },
30 return () => { window.UIFeedback = undefined; };
31 }, []);
32 return <div id="toast" className={"toast" + (msg ? " show" : "")}>{msg}</div>;
33 }
34
35 /* ----- Account Components ----- */
36 function AvatarUploader({ currentUrl }) {
37 const [file, setFile] = useState(null);
38 const upload = () => {
39 if (!file) return;
40 // Implement with Codex
41 // TODO: POST /me/avatar
42 UIFeedback.show("Avatar updated");
43 setFile(null);
44 };
45 return (
46 <div className="card panel">
47 <h2>Avatar</h2>
48 <div className="row">
49
50 <input className="file input" type="file" accept="image/*" onChange={e=>setFile(e.target.files?.[0] || null)} />
51 <button className="btn secondary" onClick={upload} disabled={!file}>Upload</button>
52 </div>
53 </div>
54);
55 }
56 }
57 </script>

```

Fig. 41: Front-End | Account

```

56 function ProfileForm({ initial }) {
57 const [name, setName] = useState(initial?.name || "");
58 const [email, setEmail] = useState(initial?.email || "");
59 const onSubmit = (e) => {
60 e.preventDefault();
61 // Implement with Codex
62 // TODO: PATCH /me
63 UIFeedback.show("Profile updated");
64 };
65 return (
66 <div className="card panel">
67 <h2>Profile</h2>
68 <form onSubmit={onSubmit}>
69 <div className="row">
70 <input className="input" placeholder="Name" value={name} onChange=
71 {e=>setName(e.target.value)} />
72 </div>
73 <div className="row">
74 <input className="input" placeholder="Email" value={email} onChange=
75 {e=>setEmail(e.target.value)} />
76 </div>
77 <button className="btn" type="submit">Save</button>
78 </form>
79 </div>
80);
81 }
82
83 function PasswordForm() {
84 const [current, setCurrent] = useState("");
85 const [next, setNext] = useState("");
86 const change = (e) => {
87 e.preventDefault();
88 if (!next) return UIFeedback.show("Enter a new password");
89 // Implement with Codex
90 // TODO: POST /me/password
91 setCurrent(""); setNext("");
92 UIFeedback.show("Password changed");
93 };
94 return (
95 <div className="card panel">
96 <h2>Change Password</h2>
97 <form onSubmit={change}>
98 <div className="row"><input className="input" type="password"
99 placeholder="Current password" value={current} onChange={e=>setCurrent(e.
100 target.value)} /></div>
101 <div className="row"><input className="input" type="password"
102 placeholder="New password" value={next} onChange={e=>setNext(e.target.
103 value)} /></div>
104 <button className="btn" type="submit">Update</button>
105 </form>
106 </div>
107);
108 }

```

Fig. 42: Front-End | Account



```

102 function ThemeToggle({ value="system" }) {
103 const [theme, setTheme] = useState(value);
104 const save = () => {
105 // Implement with Codex
106 // TODO: PATCH /me/prefs { theme }
107 document.documentElement.dataset.theme = theme;
108 UIFeedback.show("Theme saved");
109 };
110 return (
111 <div className="card panel">
112 <h2>Theme</h2>
113 <select className="select input" value={theme} onChange={e=>setTheme(e.target.
114 value)}>
115 <option value="system">System</option>
116 <option value="light">Light</option>
117 <option value="dark">Dark</option>
118 </select>
119 <div style={{marginTop:8}}><button className="btn" onClick={save}>Save</
120 button></div>
121 </div>
122);
123 }
124
125 function NotificationToggles({ prefs }) {
126 const [state, setState] = useState(prefs || { emailMarketing:false,
127 emailSecurity:true, pushGeneral:true });
128 const toggle = (k) => setState(s => ({...s, [k]: !s[k]}));
129 const save = () => {
130 // Implement with Codex
131 // TODO: PATCH /me/prefs { notifications }
132 UIFeedback.show("Preferences saved");
133 };
134 return (
135 <div className="card panel">
136 <h2>Notifications</h2>
137 <div className="row"><label><input type="checkbox" checked={state.
138 emailMarketing} onChange={()=>toggle("emailMarketing")} /> Marketing emails</
139 label></div>
140 <div className="row"><label><input type="checkbox" checked={state.
141 emailSecurity} onChange={()=>toggle("emailSecurity")} /> Security emails</
142 label></div>
143 <div className="row"><label><input type="checkbox" checked={state.pushGeneral}
144 onChange={()=>toggle("pushGeneral")} /> General push</label></div>
145 <button className="btn" onClick={save}>Save</button>
146 </div>
147);
148 }

```

Fig. 43: Front-End | Account

```

140 function ConnectedAccounts({ accounts = [] }) {
141 const providers = ["spotify", "google", "github"];
142 const isLinked = (p) => accounts.some(a => a.provider === p);
143 // Implement with Codex
144 const link = (p) => { /* TODO: POST /me/accounts/:p */ UIFeedback.show(`Linked ${p}`); };
145 // Implement with Codex
146 const unlink = (p) => { /* TODO: DELETE /me/accounts/:p */ UIFeedback.show(`Unlinked ${p}`); };
147 return (
148 <div className="card panel">
149 <h2>Connected Accounts</h2>
150 {providers.map(p => (
151 <div key={p} className="row" style={{justifyContent:"space-between"}}>
152 <div style={{textTransform:"capitalize"}}>{p}</div>
153 {isLinked(p)
154 ? <button className="btn secondary" onClick={()=>unlink(p)}>Unlink</button>
155 : <button className="btn" onClick={()=>link(p)}>Link</button>}
156 </div>
157))}
158 </div>
159);
160 }
161
162 /* ----- Account Page (tabs) ----- */
163 function AccountPage() {
164 // stubbed user; swap out once backend is wired
165 const me = {
166 name: "Karis",
167 email: "karis@example.com",
168 avatarUrl: "",
169 settings: {
170 theme: "system",
171 notifications: { emailMarketing:false, emailSecurity:true, pushGeneral:true },
172 connectedAccounts: [{ provider: "spotify", linkedAt: "2025-06-01" }],
173 },
174 };
175
176 const [tab, setTab] = useState("profile"); // 'profile' | 'password' | 'prefs' | 'connections'
177
178 const TabLink = ({id, children}) => (
179 <button className={`tab ${tab===id ? "active":""}`} onClick={()=>setTab(id)}>
180 {children}</button>
181);

```

Fig. 44: Front-End – Account

```

180 return (
181 <div className="app-container">
182 <Navbar signedIn={true} active="Account" />
183 <div className="container">
184 <div className="tabs" style={{marginTop:12}}>
185 <TabLink id="profile">Profile</TabLink>
186 <TabLink id="password">Password</TabLink>
187 <TabLink id="prefs">Preferences</TabLink>
188 <TabLink id="connections">Connected Accounts</TabLink>
189 </div>
190 </div>
191
192 <div className="app" style={{ gridTemplateColumns: "1fr" }}>
193 <main className="right-grid">
194 {tab === "profile" && (
195 <>
196 <AvatarUploader currentUrl={me.avatarUrl} />
197 <ProfileForm initial={me} />
198 </>
199)}
200 {tab === "password" && <PasswordForm />}
201 {tab === "prefs" && (
202 <>
203 <ThemeToggle value={me.settings.theme} />
204 <NotificationToggles prefs={me.settings.notifications} />
205 </>
206)}
207 {tab === "connections" && (
208 <ConnectedAccounts accounts={me.settings.connectedAccounts} />
209)}
210 </main>
211 </div>
212
213 <Toast />
214 </div>
215);
216 }
217
218 ReactDOM.createRoot(document.getElementById("root")).render(<AccountPage />);
219 </script>
220 </body>
221 </html>
222

```

Fig. 45: Front-End – Account

```

function PasswordResetPage(){
 const [email, setEmail] = useState('');
 const [newPassword, setNewPassword] = useState('');
 const [confirmPassword, setConfirmPassword] = useState('');
 const [resetCode, setResetCode] = useState('');
 const [step, setStep] = useState(1); // 1: Email, 2: Code, 3: New Password, 4: Success
 const [loading, setLoading] = useState(false);
 const [errors, setErrors] = useState({});

 function handleEmailSubmit() {
 setErrors({});
 if (!email) {
 setErrors({email: 'Email is required'});
 return;
 }
 if (!validateEmail(email)) {
 setErrors({email: 'Please enter a valid email address'});
 return;
 }

 setLoading(true);
 // Simulate API call
 setTimeout(() => {
 setLoading(false);
 setStep(2);
 }, 1000);
 }
}

```

*Fig. 47: Front-End – Password Reset*

```

function handleCodeSubmit() {
 setErrors({});
 if (!resetCode) {
 setErrors({code: 'Verification code is required'});
 return;
 }
 if (resetCode.length !== 6) {
 setErrors({code: 'Code must be 6 digits'});
 return;
 }

 setLoading(true);
 // simulate API call
 setTimeout(() => {
 setLoading(false);
 setStep(3);
 }, 1000);
}

```

*Fig. 48: Front-End - Password Reset*

```

function handlePasswordSubmit() {
 setErrors({});
 const newErrors = {};

 if (!newPassword) {
 newErrors.newPassword = 'Password is required';
 } else if (!validatePassword(newPassword)) {
 newErrors.newPassword = 'Password must be at least 8 characters, contain letters and numbers, and have no spaces';
 }

 if (!confirmPassword) {
 newErrors.confirmPassword = 'Please confirm your password';
 } else if (!passwordsMatch(newPassword, confirmPassword)) {
 newErrors.confirmPassword = 'Passwords do not match';
 }

 if (Object.keys(newErrors).length > 0) {
 setErrors(newErrors);
 return;
 }

 setLoading(true);
 // Simulate API call
 setTimeout(() => {
 setLoading(false);
 setStep(4);
 }, 1000);
}

```

Fig. 49: Front-End - Password Reset

### Test Plan

The testing plan for EchoLogz includes a diverse set of test methods to ensure core functionality and security remain through continuous development and deployment.

#### Environment

The testing plan for EchoLogz includes a diverse set of test methods to ensure core functionality and security remain through continuous development and deployment.

- Operating Systems: Windows 11/MacOS/Arch Linux
- Browser Engine: Gecko (Firefox), Chromium
- Front-end architecture: NPM, Node.js(local hosting), React.js, Vite
- Front-end testing library: Jest, React Testing Library
- Back-end architecture: FastAPI (Python)
- Back-end testing library: pytest
- DB: PostgreSQL (production), Sqlite3 (local development + testing)
- Github + Github Actions for automated pipeline testing.

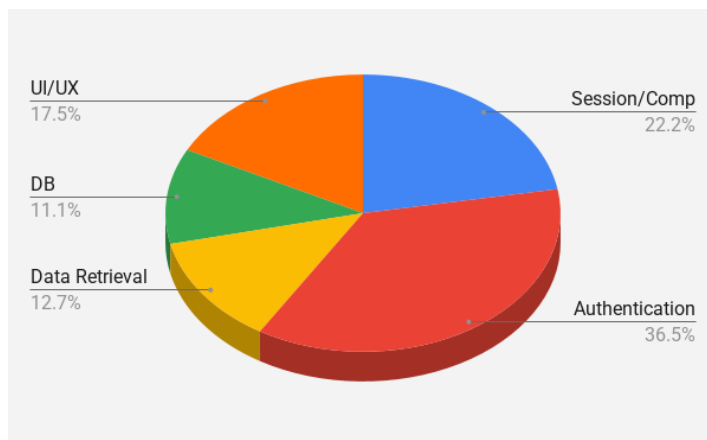


Fig. 50: Testing allocation group of all test cases

**Table 2: Hardware Requirements**

| Component             | Minimum Requirement   | Recommended           |
|-----------------------|-----------------------|-----------------------|
| <b>Processor</b>      | Intel Core I5/Ryzen 5 | Intel Core I7/Ryzen 7 |
| <b>RAM</b>            | 8 GB                  | 8GB                   |
| <b>Disk Space</b>     | 1 GB free             | 10 GB free            |
| <b>Internet Speed</b> | 10Mb/s                | 25Mb/s                |

**Table 3: Software Requirements**

| Component                  | Requirement                                               |
|----------------------------|-----------------------------------------------------------|
| <b>OS</b>                  | Windows 11/MacOS 26 Tahoe/Arch Linux                      |
| <b>Browser Engine</b>      | Gecko (Firefox)/Chromium                                  |
| <b>Front-end</b>           | npm, Node.js, React.js, Vite, Jest, React Testing Library |
| <b>Back-end</b>            | Fast API + pytest (pydantic for data validation)          |
| <b>DB</b>                  | Sqlacalehmy/Sqlite3                                       |
| <b>Automation Pipeline</b> | GitHub + GitHub Actions                                   |

## Functional Testing

### Unit Testing

- Front-end React elements
- JS functionality for client-side for user input validation
- Database components/User score data persistence
- Comparison score algorithm logic, edge cases and failures

### Integration Testing

- Front-end JS and back-end services.
  - Requests
  - Spotify OAuth
  - FastAPI

### Regression Testing

- Automated testing re-runs after new commits to master branch to protect from critical failures to ensure app health remains priority.
  - Report new bugs with non-critical features to tracking software.

## Non-Functional Testing

### Automation Testing

- Macro scripts automate testing using mock inputs to speed up development and ensure previous functionality remains intact.

### Manual Testing

- Manual stress-testing is performed to catch issues occurring from edge cases.

### Security Testing

- Safeguard common exploits
- Analyze vulnerabilities or liabilities in new developments that present potential security risks.

- Using OWASP ZAP to scan for vulnerabilities.
- JWT Validation, SQL Injection, XSS Tests
- Authentication handshakes are validation for secure encryption.
- Ensure encryption of private data

#### Performance Testing

- Analyze speed, hardware usage to identify bottlenecks.

#### UI/UX Testing:

- Manually test front-end elements for responsiveness, clarity, different display configurations and browsers.

#### Acceptance Criteria

All critical requirements listed below, functional or non-functional, must be met before deployment to ensure best user experience.

##### Functional Criteria

- Successful user account creation for a unique email address
- Successful Spotify connection with valid OAuth authentication
- Ability to create, join, or leave compare sessions.
- Spotify API calls to access necessary data for comparison algorithm.
- Successful calculation and display of compatibility scores.
- UI must provide easy site navigation with elements providing clear intentions to their functionality.
- Passing of all security tests to ensure no vulnerabilities are present in the application.

##### Non-Functional Criteria

- Performance testing
  - Focus on optimizing response times for DB queries and scoring algorithm.
- Spotify API Calls
  - Minimized and throttled to remain under rate limit.
  - Cache frequently hit endpoints to reduce calls and limit bottlenecks.
- UI
  - Readable in all forms and display sizes.
  - Comply with WGAC guidelines

#### Test Case Success Rate

- 100% of critical tests must pass
- 95% of all tests must pass before deployment

#### Test Data

##### Authentication Tests

- Valid/Invalid user account combinations
- Valid/invalid compare sessions
- Valid/invalid email verification codes
- Valid/invalid user emails for password resets
- Valid/invalid Spotify OAuth tokens

##### Unit/Security Tests

- Mock Spotify OAuth user playlist/track data

- Mock database queries
- Invalid/valid keys
- Large queries (10-100MB)
- SQL injection
- Mock user profile data

#### UI Tests

- Valid/invalid characters in fields
- Mock scoring data for graphical display
- Browser display configurations

#### Stress Tests

- Simulate hundreds of sessions created simultaneously
- Simulating hundreds of server-side compatibility score requests
- Simulating hundreds of Spotify API calls
- Simulating hundreds of user account requests

#### Test Tables

##### Authentication/Accounts

| Test ID         | Test Case Description                | Expected Outcome                                                           | Status      |
|-----------------|--------------------------------------|----------------------------------------------------------------------------|-------------|
| <b>TFR10-01</b> | User registration                    | Account creation;<br>User emailed;                                         | Not Started |
| <b>TFR10-02</b> | Users can reset passwords            | Account verification;<br>User emailed if exists;<br>Update user password;  | Not Started |
| <b>TFR10-03</b> | Users can update account information | Email; Password; First Name; Last Name; Spotify Conn;                      | Not Started |
| <b>TFR10-04</b> | User login with JWT                  | Valid credentials                                                          | Not Started |
| <b>TFR10-05</b> | Reject duplicate usernames           | Raise HTTP error                                                           | Not Started |
| <b>TFR10-06</b> | Validate account input data          | Bad request for empty or required fields                                   | Not Started |
| <b>TFR10-07</b> | Reject incorrect credentials         | Bad request                                                                | Not Started |
| <b>TFR10-08</b> | Reject login for unknown users       | Bad request                                                                | Not Started |
| <b>TNFR5-01</b> | Test Password Encryption             | Passwords are hashed securely                                              | Not Started |
| <b>TNFR4-01</b> | Test JWT Validation                  | Valid tokens are accepted; Invalid or expired tokens result in bad request | Not Started |



|                 |                        |                                                        |             |
|-----------------|------------------------|--------------------------------------------------------|-------------|
| <b>TFR10-09</b> | Test input validation  | Usernames, passwords, email, SQL injection             | Not Started |
| <b>TFR10-10</b> | Test edge cases        | Case sensitivity, Unicode, concurrent signups, retries | Not Started |
| <b>TFR10-11</b> | Signup integration     | Full signup -> login -> /account flow                  | Not Started |
| <b>TFR10-12</b> | Spotify login redirect | Redirect to Spotify OAuth                              | Not Started |

*Table 4: Authentication and Accounts*

#### Data Retrieval

| <b>Test ID</b> | <b>Test Case Description</b>               | <b>Expected Outcome</b>                                                                    | <b>Status</b> |
|----------------|--------------------------------------------|--------------------------------------------------------------------------------------------|---------------|
| <b>TFR3-01</b> | Retrieve tracks from valid public playlist | Return tracks list                                                                         | Not Started   |
| <b>TFR3-02</b> | Handle empty playlist                      | Returns empty list                                                                         | Not Started   |
| <b>TFR3-03</b> | Playlist unavailable                       | Handle HTTP error from Spotify                                                             | Not Started   |
| <b>TFR3-04</b> | Spotify rate limit (429)                   | Proper error response                                                                      | Not Started   |
| <b>TFR3-05</b> | User top tracks                            | Raise error for invalid params; Success for valid response; Empty list for NaN Json return | Not Started   |
| <b>TFR3-06</b> | User top artists                           | Raise error for invalid params; Success for valid response; Empty list for NaN Json return | Not Started   |
| <b>TFR9-01</b> | Retrieve audio features                    | Requested feature set returned                                                             | Not Started   |
| <b>TFR9-02</b> | Missing metadata handling                  | Partial data still retrieved                                                               | Not Started   |

*Table 5: Data Retrieval*

#### Database

| Test ID  | Test Case Description                       | Expected Outcome                             | Status      |
|----------|---------------------------------------------|----------------------------------------------|-------------|
| TFR5-01  | Persist musical identity scores             | Identity scores saved to DB                  | Not Started |
| TFR5-02  | Identity scores are saved to DB             | DB contains record of score                  | Not Started |
| TFR5-03  | Recalculate scores on updated playlist      | Identity scores updated                      | Not Started |
| TFR5-04  | Missing audio features in score calculation | Graceful handling for missing audio features | Not Started |
| TFR13-01 | EchoLogz connects to DB                     | Connection successful                        | Not Started |
| TFR13-02 | App handles disconnect                      | Reconnect or raise error                     | Not Started |
| TFR13-03 | Read/write latency                          | DB processes data within required threshold  | Not Started |

Table 6: Database

## UI/UX

| Test ID  | Test Case Description                | Expected Outcome                                 | Status      |
|----------|--------------------------------------|--------------------------------------------------|-------------|
| TFR7-01  | Show user profile                    | User profile displays when signed in;            | Not Started |
| TFR7-02  | Render matches list                  | Shows list of matches;                           | Not Started |
| TFR7-03  | Missing profile data                 | Method to gracefully handle missing profile data | Not Started |
| TFR7-04  | Slow network performance             | UI handles slower networks                       | Not Started |
| TFR11-01 | Display compatibility visualization  | UI loads compatibility chart                     | Not Started |
| TFR11-02 | Visualization matches backend values | UI displays proper values from backend           | Not Started |
| TFR11-03 | Handle invalid or NULL data          | Method for graceful fallback                     | Not Started |
| TFR11-04 | Browser compatibility                | Same rendering across major browsers (Chrome,    | Not Started |

|                 |                                        |                                                 |             |
|-----------------|----------------------------------------|-------------------------------------------------|-------------|
|                 |                                        | Firefox, Opera, Edge, Safari)                   |             |
| <b>TFR14-01</b> | UI interface loads on mobile & desktop | Proper layout; Navigate easily and successfully | Not Started |
| <b>TFR14-02</b> | Functional buttons and links           | Correct navigation on site                      | Not Started |
| <b>TFR14-03</b> | Accessibility                          | Contains basic accessibility options            | Not Started |

Table 7: UI/UX

### Test Scripts

```

1 /**
2 * @jest-environment jsdom
3 */
4
5 import { render, screen } from "@testing-library/react";
6 import Navbar from "../js/navbar.jsx";
7
8 test("Render sign up/sign in text when signed out", () => {
9 render(<Navbar signedIn={false} username="" active="" />);
10 expect(screen.getByText("Sign In")).toBeInTheDocument();
11 expect(screen.getByText("Sign Up")).toBeInTheDocument();
12 });
13
14 test("Render user profile links when signed in", () => {
15 render(<Navbar signedIn={true} username="TestUser" active="" />);
16 expect(screen.getByText("Dashboard")).toBeInTheDocument();
17 expect(screen.getByText("Saved")).toBeInTheDocument();
18 expect(screen.getByText("Account")).toBeInTheDocument();
19 expect(screen.getByText("Sign Out")).toBeInTheDocument();
20 });
21
22 test("Apply correct active class", () => {
23 render(<Navbar signedIn={true} username="TestUser" active="Dashboard" />);
24 const dashboard = screen.getByText("Dashboard");
25 expect(dashboard.classList.contains("active")).toBe(true);
26 });
27

```

Fig. 51: Test Script: Frontend Unit Testing

```
1 import { validateEmail, validatePassword, passwordsMatch } from "../js/utils.js";
2
3 test("Ensure validateEmail accepts valid email format", () => {
4 expect(validateEmail("test@example.com")).toBe(true);
5 });
6
7 test("Ensure validateEmail rejects invalid email format", () => {
8 expect(validateEmail("bad-email")).toBe(false);
9 expect(validateEmail("a@b")).toBe(false);
10 });
11
12 test("Ensure validatePassword only accepts alphanumeric chars under max length", () => {
13 expect(validatePassword("short1")).toBe(false);
14 expect(validatePassword("has space 1")).toBe(false);
15 expect(validatePassword("allletters")).toBe(false);
16 expect(validatePassword("12345678")).toBe(false);
17 expect(validatePassword("Valid123")).toBe(true);
18 });
19
20 test("Ensure passwordsMatch evaluates correctly", () => {
21 expect(passwordsMatch("abc", "abc")).toBe(true);
22 expect(passwordsMatch("abc", "abd")).toBe(false);
23 });
24
```

Fig. 52: Test Script: Frontend Unit Testing

```

""" BACKEND (FastAPI + PyTest) UNIT TEST
IMPORTS:
TestClient: let's you fake HTTP requests to your FastAPI app
"""

from fastapi.testclient import TestClient
from backend.main import app
from backend.echoDB import db_schemas, db_session, db_crud
from backend.core.security import hash_password

client = TestClient(app) # fake API client

test_user = ''
test_password = ''
wrong_password = ''

def setup_test_user():
 db = db_session.SessionLocal()
 db.query(db_schemas.User).delete() # Clean DB first
 fake_user = {
 "username": test_user,
 "email": "test@example.com",
 "password": test_password
 }
 # Create test user w/ real CRUD function
 hashed = hash_password(fake_user["password"])
 db_crud.create_user_with_hash(
 db=db,
 username=fake_user["username"],
 email=fake_user["email"],
 hashed_pw=hashed
)
 db.close()

```

Fig. 53: Test Script: Backend Unit Testing

```
def test_login_success() -> None:
 payload = {
 "username": test_user,
 "password": test_password,
 } # real clients can't send JSON - this is just feature of TestClient + Starlette
 # REAL CLIENT FORMAT: username=test_user&password=test_password
 response = client.post("/auth/login", json=payload) # Send post request to "auth/login"
 assert response.status_code == 200
 body = response.json()
 assert "access_token" in body
 assert body.get("token_type", "").lower() == "bearer"

def test_login_fail_wrong_pw() -> None:
 payload = {
 "username": test_user,
 "password": wrong_password,
 }
 response = client.post("/auth/login", json=payload)
 assert response.status_code == 401
 assert response.json() == {"detail": "Bad credentials"}
```

*Fig. 54: Test Script: Backend Unit Testing*

## Test Report

To evaluate potential security weaknesses in the EchoLogz application, an automated vulnerability scan was performed using **OWASP ZAP 2.16.1**. The scan targeted the local development instance (<http://localhost:5173/>) and analyzed the frontend for common vulnerabilities including insecure headers, path exposure, and improper asset disclosure.

ZAP identified several issues typical of development builds, which are outlined below:

### 1. Missing Content Security Policy (CSP) Header

- **Severity:** Medium
- **Description:** The application did not return a CSP header, which can increase exposure to cross-site scripting (XSS) or content injection attacks.
- **Cause:** Vite's development server does not inject CSP headers by default.
- **Mitigation (Production):** Add CSP headers using reverse proxy configuration (e.g., Nginx) or FastAPI middleware to restrict allowed scripts, frames, and sources.

### 2. Hidden File Exposure

- **Severity:** Low

- **Description:** ZAP detected access to files such as .bzip/, .hg/, and sitemap.xml, which may reveal metadata or development information.
- **Cause:** These files are not actually part of EchoLogz's codebase; rather, they were discovered due to ZAP crawling default dev server paths.
- **Mitigation (Production):** Production builds will not expose source-mapped development folders. These paths are not present once the React app is built and deployed.

### 3. Path Traversal Warning

- **Severity:** Low
- **Description:** ZAP flagged a potential path traversal risk when accessing internal Vite-generated asset directories (e.g., /node\_modules/.vite/deps/react.js).
- **Cause:** The development server makes transpiled module bundles visible for hot reloading.
- **Mitigation (Production):** These paths are removed during bundling. Static assets are minimized and served from a locked-down directory with no user-accessible source maps.

### 4. Modern Web Application Notice (Informational)

- **Severity:** Informational
- **Description:** ZAP recognized the structure of the application as a modern SPA with dynamically loaded JavaScript modules.
- **Impact:** This is not a vulnerability; it simply indicates that modern exploration techniques (e.g., AJAX spider) work best for scanning.
- **Mitigation:** No action required.

Overall, the ZAP results did not reveal any critical vulnerabilities within the EchoLogz application. Most findings stemmed from expected behavior of the Vite development server and do not apply to the production deployment environment. In a production setting, EchoLogz would enable CSP headers, disable directory listings, restrict static asset exposure, and serve the compiled React build behind a secure FastAPI or Nginx layer.

### Team Assignments and Responsibilities

**Karis Land**, as the **Project Manager**, contributed across every layer of EchoLogz by helping design, build, and refine core backend and frontend components while keeping the team aligned and moving. She balanced workload across the group, managed timelines, maintained communication channels and shared documentation, enforced ethical and legal standards, and made sure deadlines were met. On the technical side, she worked closely with teammates on the FastAPI backend, implementing key pieces of the authentication system, the email-verification workflow, and the Spotify OAuth 2.0 integration. She also collaborated on the database layer, contributing to SQLAlchemy models, schemas, and CRUD operations. On the frontend, she helped convert mockups into functional React components, integrated protected routing, established major frontend-to-backend pipelines, and implemented the Spotify Connect experience. Throughout development, she focused on high-level system design, real-time debugging, improving code structure, and writing clear module documentation to keep the team synced and the full stack behaving consistently.

**Duncan Howard**, as the **UI Director and Testing Guru**, played a key role in shaping how EchoLogz looks, feels, and proves its correctness. He focused early on planning the design and implementation strategies for the graphical interface, ensuring that the UI would stay intuitive, consistent, and aligned with the project's overall design direction. Alongside UI planning, he coordinated with the rest of the team on how the Rust backend modules and the broader testbed would integrate with the frontend experience, keeping the visual layer connected to the underlying system behavior. As the project moved into heavier implementation, Duncan drafted the overall testing plan for EchoLogz, making sure each requirement had a clear validation path and that the system could be tested reliably. He also contributed directly to front-end stability by writing React.js unit tests using Jest, reinforcing accuracy and structure of the components as they were developed. Throughout development, he supported the goal of delivering an application that is both visually appealing and consistently functional, ensuring that UI decisions and testing workflows stayed aligned with the team's broader objectives.

**April Carr**, as the **Technical Documentation Lead**, shaped the written backbone of EchoLogz by developing and maintaining the project's documentation from the beginning of development through every major system change. She ensured that requirements, design decisions, and evolving system behaviors were captured clearly and consistently so the entire team stayed aligned as the platform matured. As the project progressed, she expanded her responsibilities beyond documentation, researching and analyzing existing market tools similar to EchoLogz to help refine the system's direction and feature expectations. April also contributed directly to implementation by designing and building



both frontend and backend components for the profiles and settings experience. As development moved into heavier testing and backend expansion, she stepped into the Technical Documentation and QA Lead role, designing test cases for the RTMatrix and building the Spotify feature-vector module that supports core backend scoring functionality. Throughout the project, she kept documentation accurate, supported QA planning, and contributed to critical technical components, helping the system evolve cleanly across sprints.

**Austin Johnson**, as the **Technical/Standards Coordinator**, focused on monitoring and evaluating all functional and non-functional requirements to keep the system's scope realistic and deliverable within each sprint. He ensured that planned modules aligned with project constraints and that development stayed grounded in what could actually be built on time. As the project advanced, Austin expanded this oversight into hands-on contributions by developing the comprehensive testing plan for EchoLogz, covering major functionality as well as critical edge cases to ensure reliability across the app. He also refactored frontend components to support clean navigation between the home, sign-in, and sign-up pages, improving the clarity and flow of the early UI. On the backend, he contributed to the Spotify call module and the scoring module, strengthening the core logic that powers playlist retrieval and compatibility scoring. Throughout development, he supported both standards enforcement and technical execution, helping keep the project stable, testable, and aligned with its stated requirements.

**Jaxsen Honeycutt**, as the **Debugging Specialist and Artistic Director**, supported the project by contributing to both the functional and non-functional requirements, researching project ideas, and helping troubleshoot early constraints that shaped the direction of EchoLogz. Throughout development, he continued assisting with requirement clarification and constraint resolution, offering input that strengthened the system's foundational planning. As implementation progressed, Jaxsen provided additional support by clarifying documentation to improve team understanding and expanding the initialized React project, helping establish a clearer structure for future UI development. His role centered on smoothing technical and conceptual friction points while contributing to the project's visual and structural setup.